

机器视觉基础

孙明竹 刘晓芳

sunmz@nankai.edu.cn

liuxiaofang@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

Nankai University

第二部分 图像处理基础

Image Processing

主要内容

- 第2章 数字图像的概念与性质
- 第3章 图像预处理
- 第4章 二值图像处理
 - 4.1 图像阈值化 (6.1节)
 - 4.2 二值图像边界跟踪 (6.2.3节)
 - 4.3 二值图像形态学 (13.1节, 13.3节, 13.5节)
- 第5章 传统特征检测

4.1 图像阈值化

- 图像阈值化属于图像分割，分割的目标：将图像划分为与其中含有的真实世界的物体或区域有强相关性的组成部分
- 分割分类
 - 完全分割：分割结果是一组唯一对应于输入图像中物体的互不相交的区域集合
 - 部分分割：分割后的区域并不直接对应于物体，图像被划分一些同性质区域

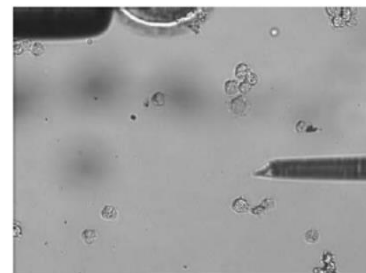


Fig. 3: Somatic cells in operation

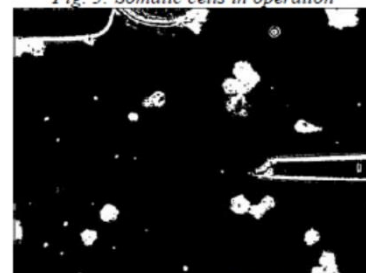


Fig. 4: Texture segmentation result of somatic cell image

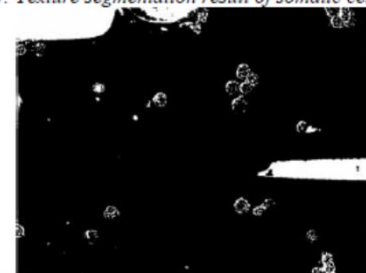


Fig. 5: OTSU segmentation result of somatic cell image

4.1 图像阈值化

- 完全分割 <https://segment-anything.com/>



4.1 图像阈值化

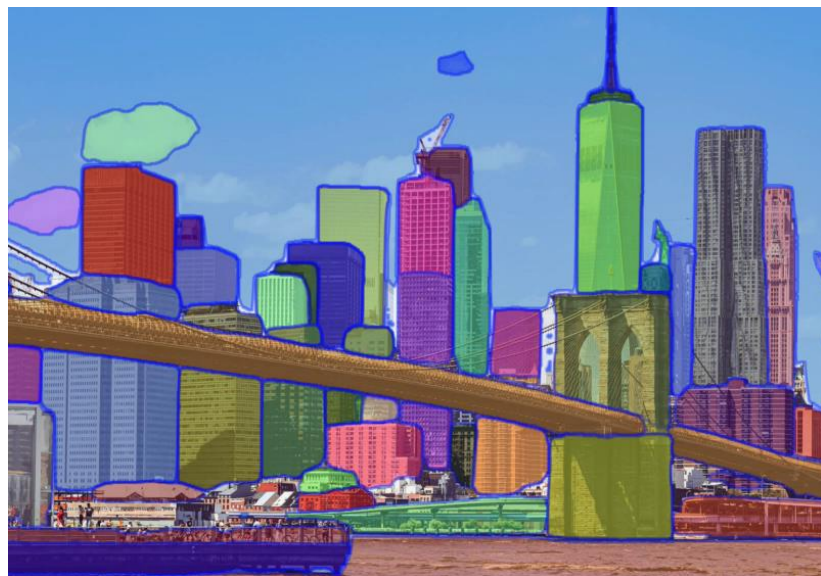
- 图像R的完全分割是区域 $R_1, \dots, R_S$ 的有限集合:

$$R = \bigcup_{i=1}^S R_i \quad R_i \cap R_j = \emptyset \quad i \neq j$$

- 阈值化是输入图像f到输出图像g的变换:

$$g(m, n) = \begin{cases} 1 & f(m, n) \geq T \\ 0 & f(m, n) \leq T \end{cases}$$

- 选择正确的阈值是阈值分割成功的关键



4.1 图像阈值化

➤ 各种对阈值进行修正的方法:

- 局部阈值化: 阈值是局部图像特征的函数, 在图像范围内是变化的, 即: $T = T(f, f_c)$

- 带(band)阈值化:

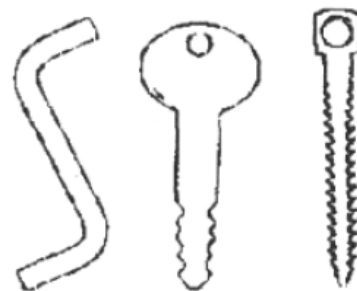
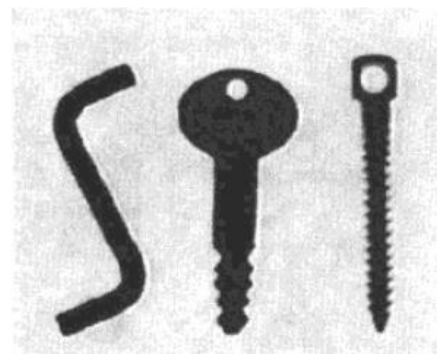
$$g(m, n) = \begin{cases} 1 & f(m, n) \in D \\ 0 & \text{else} \end{cases}$$

- 多阈值的阈值化:

$$g(m, n) = \begin{cases} 1 & f(m, n) \in D_1 \\ 2 & f(m, n) \in D_2 \\ \dots & \\ n & f(m, n) \in D_n \\ 0 & \text{else} \end{cases}$$

- 半阈值化:

$$g(m, n) = \begin{cases} f(m, n) & f(m, n) \geq T \\ 0 & f(m, n) \leq T \end{cases}$$



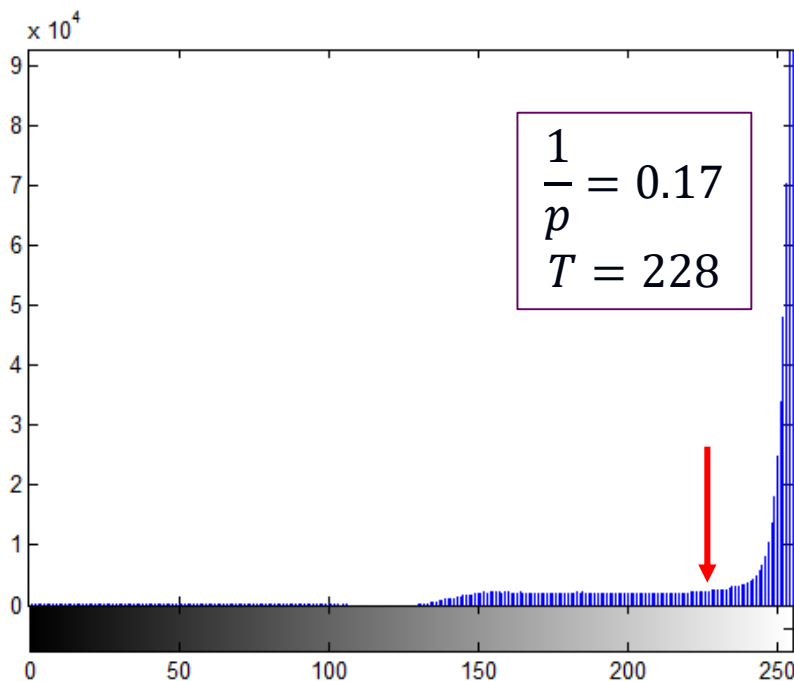
4.1.1 阈值检测方法

➤ p率阈值化

- 举例：印刷文本上字符覆盖 $1/p$ 的纸张面积
- 基于图像直方图选择阈值 T ，使得 $1/p$ 的图像面积具有比 T 小的灰度值

如果事先知道图像分割的某种性质，就可以简化阈值选择的任务的条件来选择。一个例子是我们可以知道在印刷文本页上文本字符覆盖很容易选择一个阈值 T （基于图像的直方图）使得 $1/p$ 的图像面积具有这种方法称为 p 率阈值化（ p -tile thresholding）。不幸的是，通常我们信息。

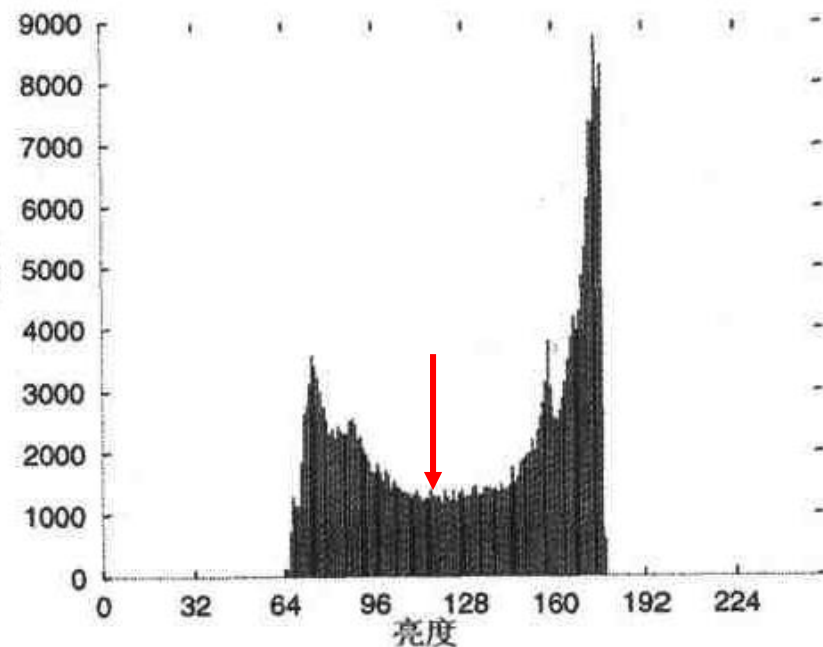
更为复杂的阈值检测方法是基于直方图形状分析的。如果图像由的物体所组成，所产生的直方图是二模态的。物体像素构成其中的图 6.3 给出了一些典型的例子。直方图形状说明了一个事实，即两个物体和背景间的边界造成的。阈值应该满足最小分割错误的要求——直方图数值的那个灰度值作为阈值。如果直方图是多模态的，在任意多阈值；每个阈值当然会给出不同的分割结果。式(6.6)给出的多



4.1.1 阈值检测方法

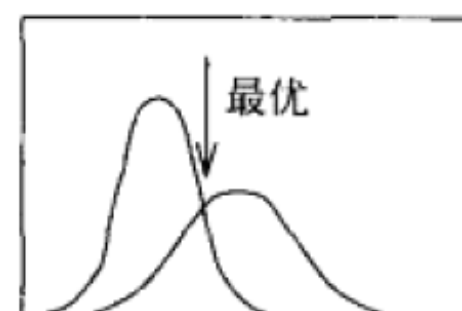
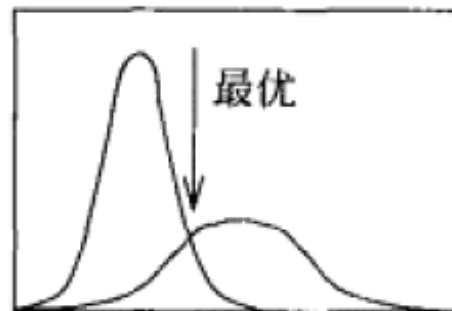
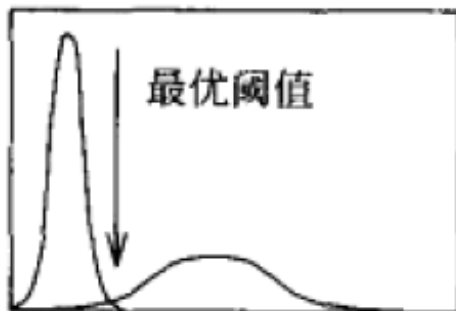
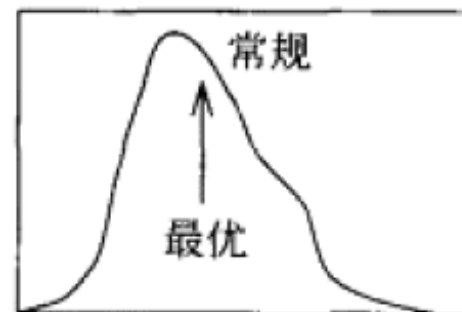
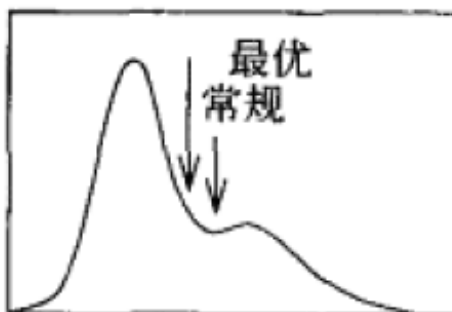
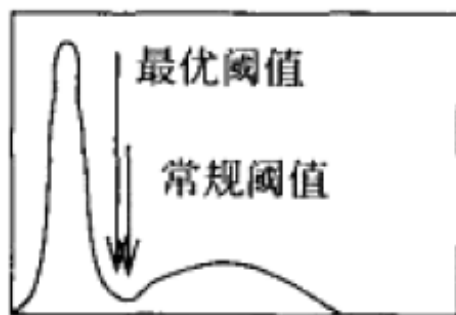
➤ 基于直方图的形状分析阈值

- 二模态直方图：图像由不同于背景灰度值、且具有近似相同灰度的物体组成，产生二模态直方图
- 物体像素和背景像素各构成一个峰，选取两个峰之间频率最小的灰度值作为阈值
- 模式方法(mode method)：寻找直方图的两个局部极大值，取它们之间的极小值作为阈值



4.1.2 最优阈值化

- 将图像的直方图用两个或多个正态分布的概率密度函数来近似
- 获取多个正态分布最大值之间的最小概率处，取距离该处最近的灰度值作为阈值



4.1.2 最优阈值化

算法：迭代的（最优）阈值选择

1. 设定初始估计阈值 $t = t_0$ ，可将 t_0 设为图像中最大亮度值与最小亮度值的中间值
2. 使用阈值 t 分割图像，设亮度值 $\geq t$ 的所有像素组成集合 G_1 ，亮度值 $< t$ 的所有像素组成集合 G_2
3. 分别计算 G_1 和 G_2 内像素的平均灰度值 μ_1 和 μ_2 ：
$$\mu_1 = \frac{\sum_{(i,j) \in G_1} f(i,j)}{\#G_1_pixels} \quad \mu_2 = \frac{\sum_{(i,j) \in G_2} f(i,j)}{\#G_2_pixels}$$
4. 计算一个新阈值 $t = (\mu_1 + \mu_2)/2$
5. 重复步骤2到步骤4，直到连续迭代中阈值 t 的变化足够小

4.1.2 最优阈值化

算法：OTSU阈值检测

1. 给定包含G个灰度级的图像I，图像的归一化直方图为：

$$H(i) = n_i / N \quad i = 0, 1, 2, \dots, G-1$$

2. 对于每个可能的阈值t，将直方图分为灰度级为[0, 1, ..., t]的像素集合（背景B）和灰度级为[t+1, ..., G-1]的像素集合（前景F）

3. 计算一个像素是背景/前景的概率：

$$\omega_B(t) = \sum_{i=0}^t H(i) \quad \omega_F(t) = \sum_{i=t+1}^{G-1} H(i)$$

$$\mu_B(t) = \sum_{i=0}^t i \frac{H(i)}{\omega_B}$$

4. 计算背景和前景的类间方差

$$\sigma(t) = \omega_B(t)(\mu_B(t) - \mu)^2 + \omega_F(t)(\mu_F(t) - \mu)^2$$

$$\mu_F(t) = \sum_{i=t+1}^{G-1} i \frac{H(i)}{\omega_F}$$

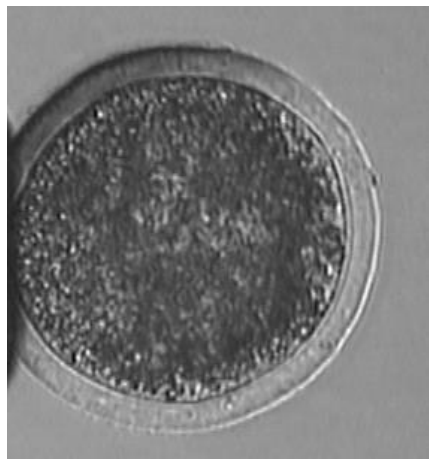
5. 选择最优阈值为 $\hat{t} = \arg \max_t (\sigma(t))$

$$\mu = \sum_{i=0}^{G-1} i H(i)$$

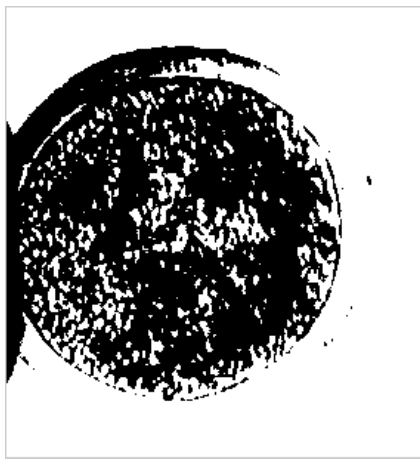
阈值化方法对比

➤ OTSU阈值化

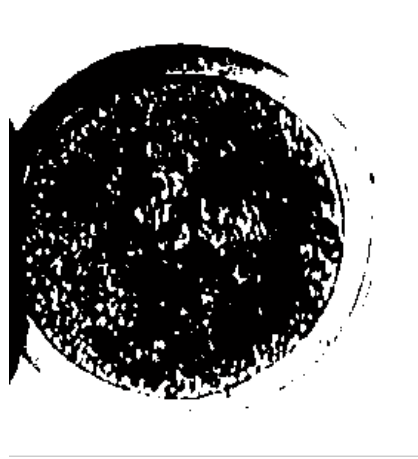
- (Matlab:graythresh)
- (OpenCV:threshold(..., THRESH_OTSU))



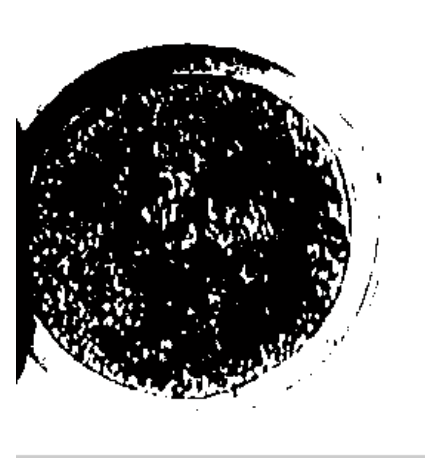
原始细胞图像



固定阈值
t=100



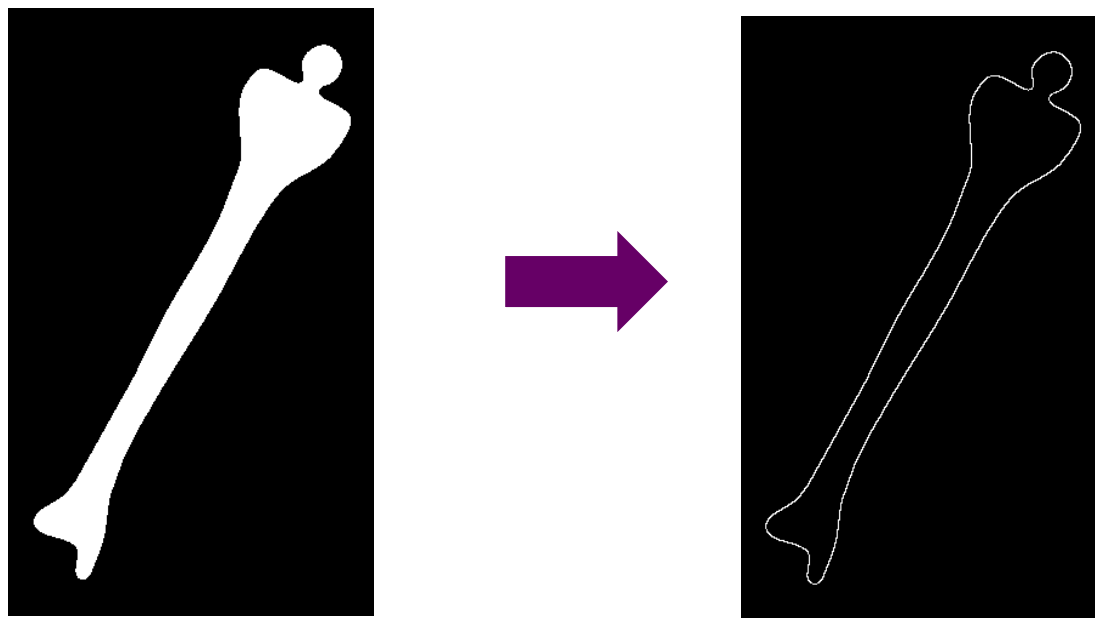
自动阈值选择
t=113



OTSU二值化
t=113

4.2 二值图像边界跟踪

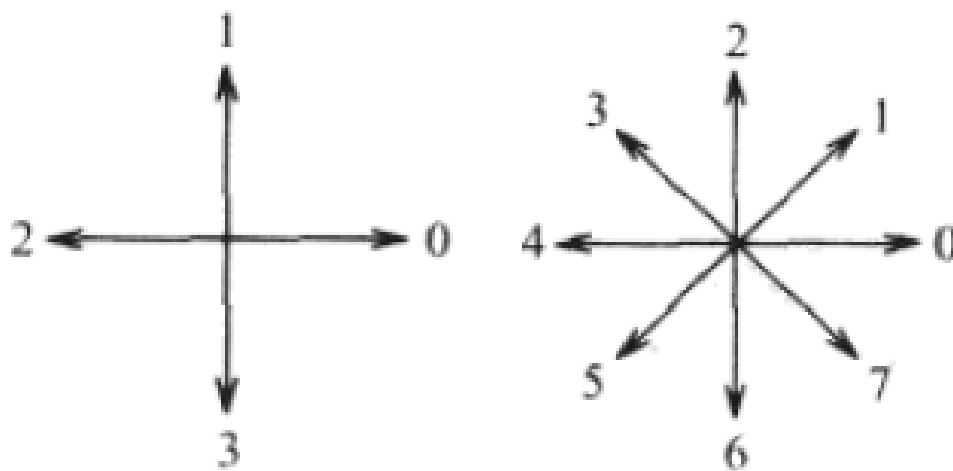
- 在二值图像或区域已经标注的图像中获取边界
 - 内(Inner)边界：区域内边界是区域的一个子集
 - 外(Outer)边界：外边界不是区域的一个子集



4.2 二值图像边界跟踪

算法：内边界跟踪

1. 从左上方开始搜索图像，直到找到一个新区域的一个像素 P_0 ， P_0 是区域边界的起始像素
2. 定义变量 dir ，存储从前一个边界元素到当前边界元素的前一个移动方向，置 $dir=3$ (4-邻接跟踪) / 7 (8-邻接跟踪)



算法：内边界跟踪（续）

3. 按照逆时针方向搜索当前像素的 3×3 邻域，从以下的方向开始搜索邻域：

4-邻接跟踪： $(dir+3) \bmod 4$

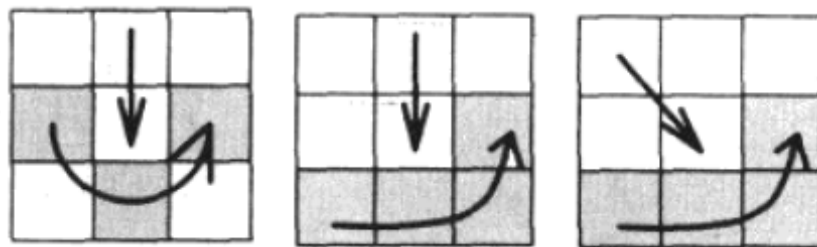
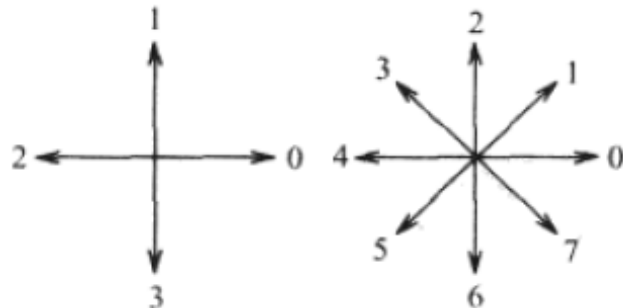
8-邻接跟踪： $(dir+7) \bmod 8$ (dir 为偶数)

$(dir+6) \bmod 8$ (dir 为奇数)

找到的第一个与当前像素值相同的像素是一个新的边界元素 P_n ，更新 dir 数值

4. 如果当前的边界元素 P_n 等于第二个边界元素 P_1 ，且前一个边界元素 P_{n-1} 等于 P_0 ，则停止；否则，重复第3步

5. 检测到的内边界由像素 $P_0, \dots, P_{n-2}$ 构成

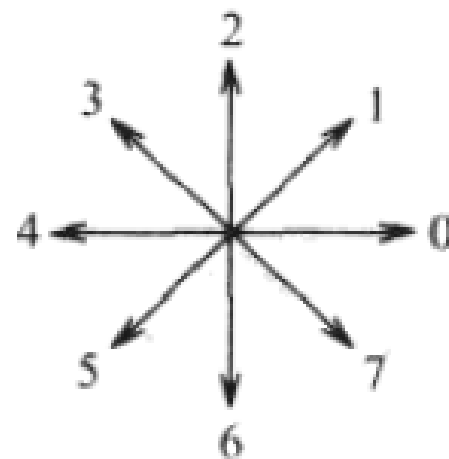
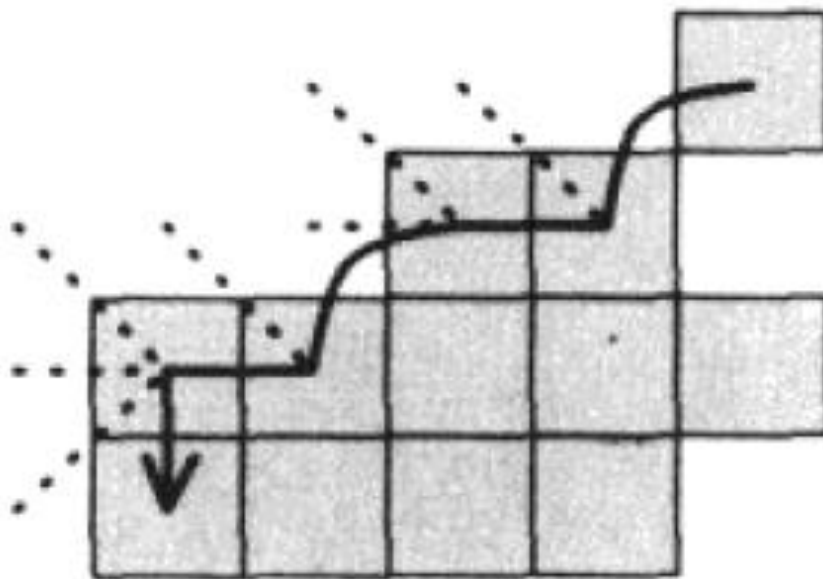


4.2 二值图像边界跟踪

➤ 内边界跟踪

8-邻接跟踪: $(dir+7) \bmod 8$ (dir为偶数)

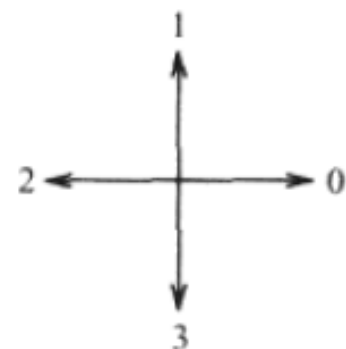
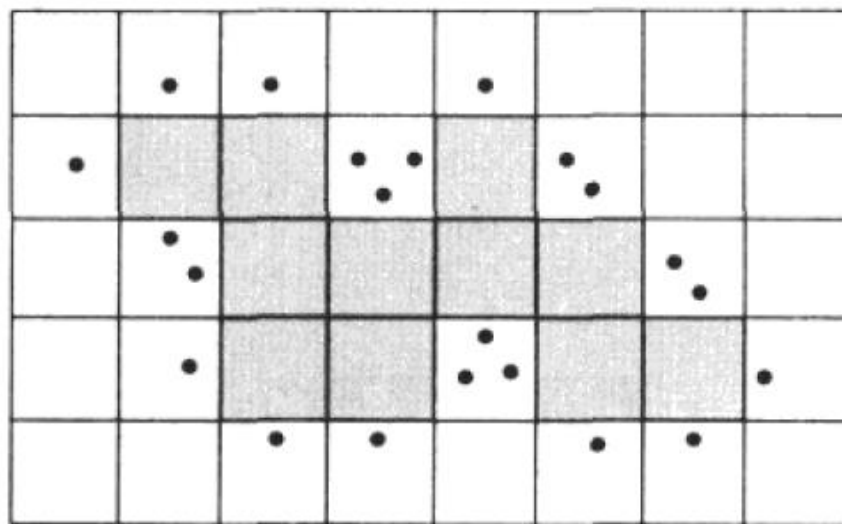
$(dir+6) \bmod 8$ (dir为奇数)



4.2 二值图像边界跟踪

算法：外边界跟踪

1. 根据4-邻接跟踪区域内边界
2. 外边界由所有在搜索过程中测试过的非区域像素组成，如果某些像素被测试超过一次，它们就被在外边界上列出超过一次



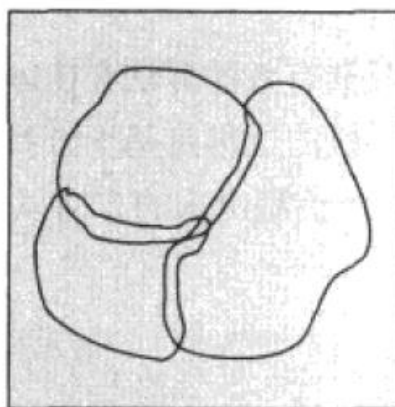
4.2 二值图像边界跟踪

➤ 扩展边界

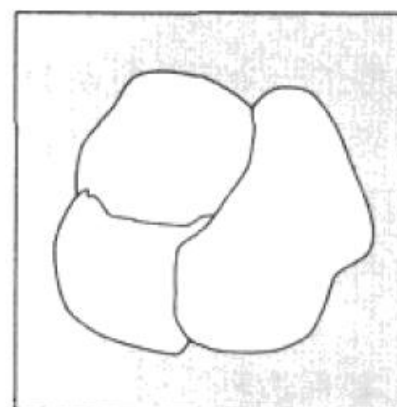
- 定义了相邻区域之间存在的单一公共边界，可以使用标准像素坐标来描述
- 保留了外边界有用的性质
- 边界形状与内边界完全相同，只是边界像素位置分别向下和向右平移了半个像素



内边界



外边界



扩展边界

4.2 二值图像边界跟踪

3	2	1
4	P	0
5	6	7

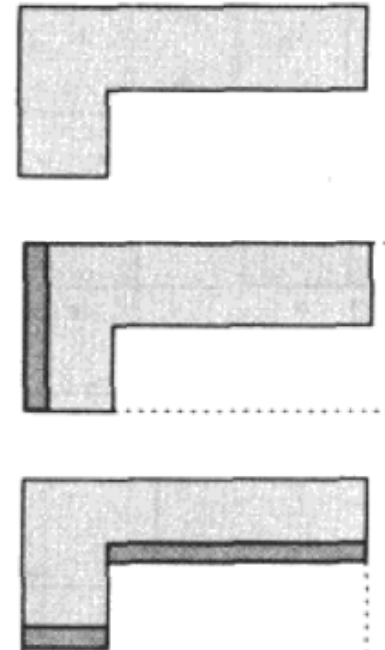
➤ 扩展边界用8-邻接定义

- $P_4(P)$ 表示像素P直接左像素， $P_0(P)$ 表示像素P直接右像素
- $P_2(P)$ 表示像素P直接上像素， $P_6(P)$ 表示像素P直接下像素

➤ 区域R的内边界像素

如果 $P_4(P) \in Q$ ，则 P 是 R 的左像素，标识为 $LEFT(R)$
 如果 $P_0(P) \in Q$ ，则 P 是 R 的右像素，标识为 $RIGHT(R)$
 如果 $P_2(P) \in Q$ ，则 P 是 R 的上像素，标识为 $UPPER(R)$
 如果 $P_6(P) \in Q$ ，则 P 是 R 的下像素，标识为 $LOWER(R)$

(Q 代表区域 R 外的像素)



4.2 二值图像边界跟踪

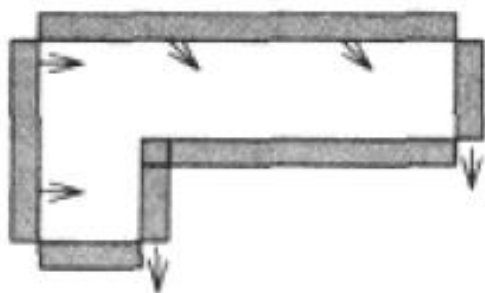
- 扩展边界EB定义为满足下述条件的像素 P, P_0, P_6, P_7 的集合

$$\begin{aligned}
 EB = & \{P : P \in LEFT(R)\} \cup \\
 & \{P : P \in UPPER(R)\} \cup \\
 & \{P_6(P) : P \in LOWER(R)\} \cup \\
 & \{P_6(P) : P \in LEFT(R)\} \cup \\
 & \{P_0(P) : P \in RIGHT(R)\} \cup \\
 & \{P_7(P) : P \in RIGHT(R)\}
 \end{aligned}$$

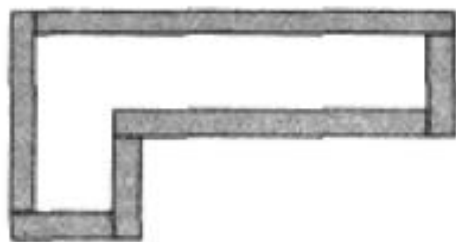
3	2	1
4	P	0
5	6	7

4.2 二值图像边界跟踪

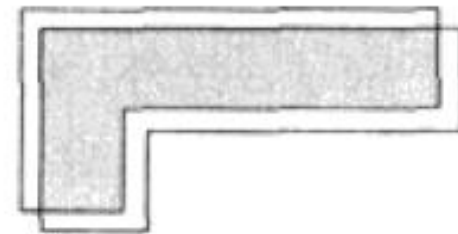
- 从外边界建立扩展边界
 - 向下和右移位所有的UPPER外边界点一个像素
 - 向右移位所有的LEFT外边界点一个像素
 - 向下移位所有的RIGHT外边界点一个像素
 - LOWER外边界保持不变



外边界



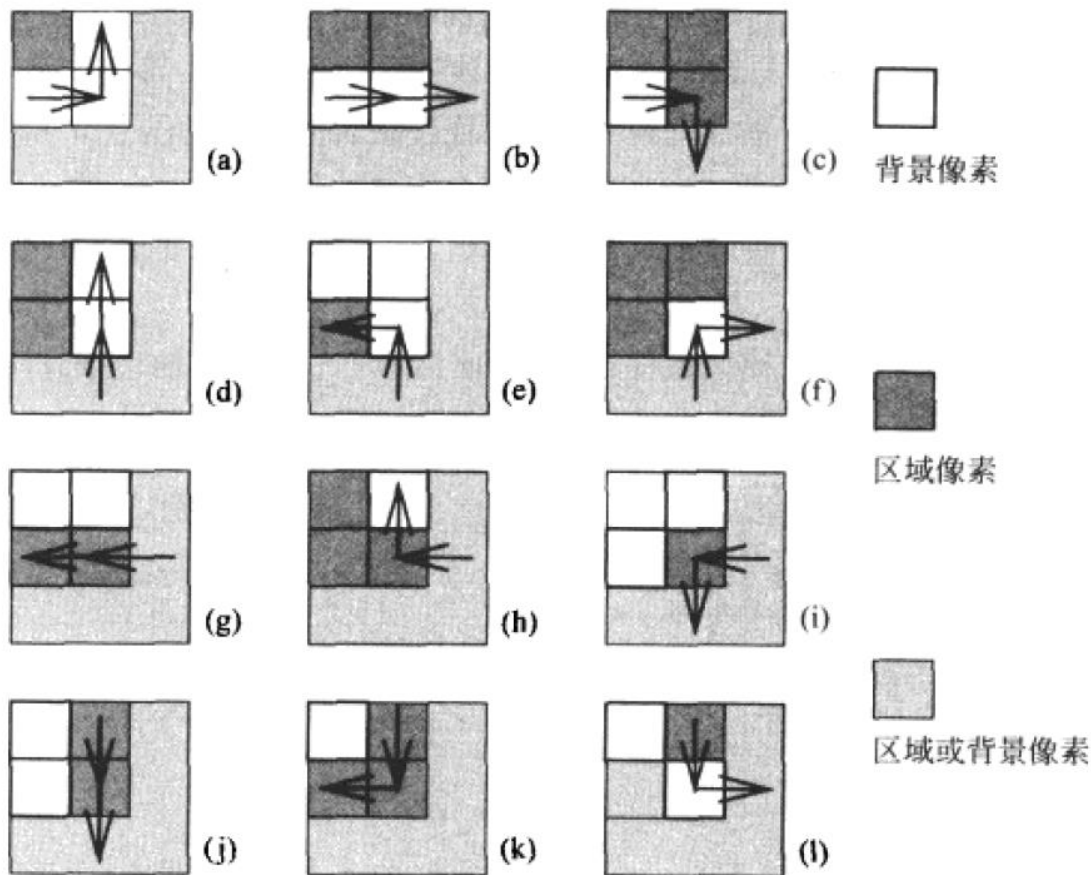
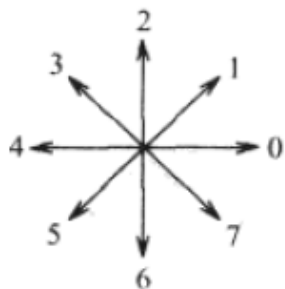
扩展边界



扩展边界与自然
物体边界具有
相同的形状与尺寸

基于查找表的扩展边界跟踪 *

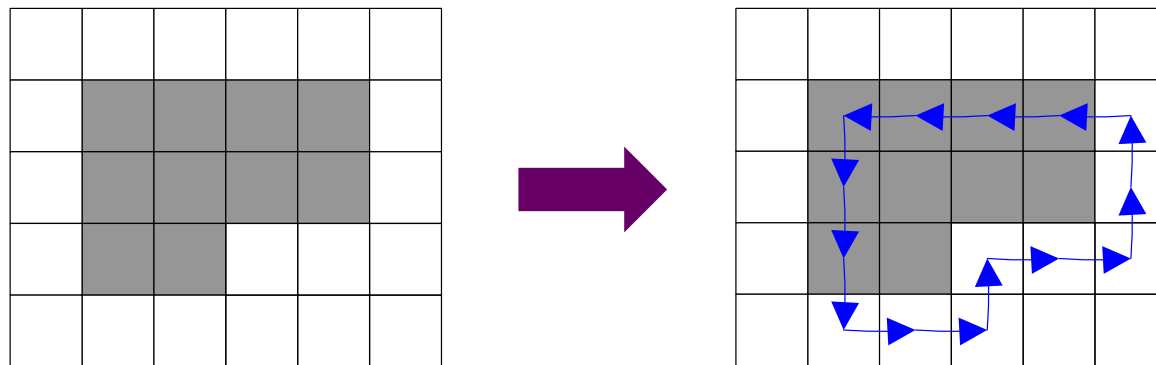
- 依据边界的前一个检测方向及窗口像素的状态确定12种可能的情况
- 窗口像素状态可以是区域的内部或外部



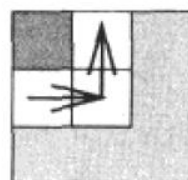
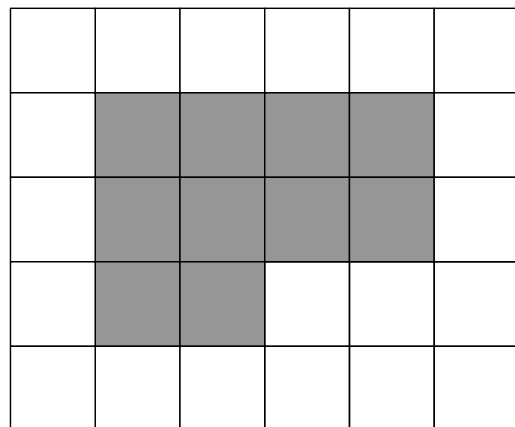
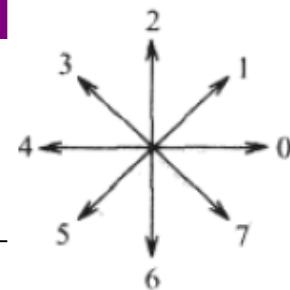
基于查找表的扩展边界跟踪 *

算法：基于查找表的扩展边界跟踪

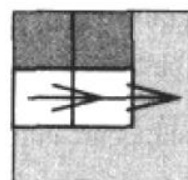
1. 按照标准方法，从左上方开始搜索图像，直到找到一个新区域的一个像素作为起始像素
2. 从起始像素沿着跟踪边界的第一个移动方向是 $\text{dir}=6$ （向下），对应情况(i)
3. 按照查找表跟踪扩展边界，直到得到一个封闭的扩展边界



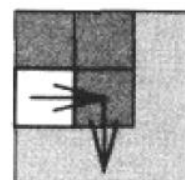
基于查找表的扩展边界跟踪 *



(a)



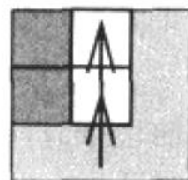
(b)



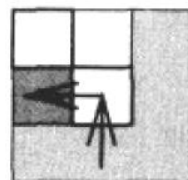
(c)



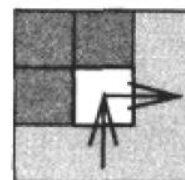
背景像素



(d)



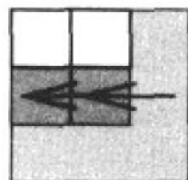
(e)



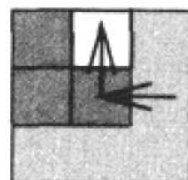
(f)



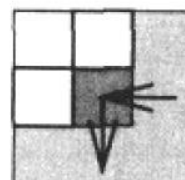
区域像素



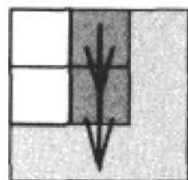
(g)



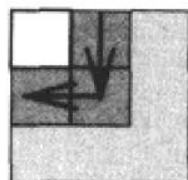
(h)



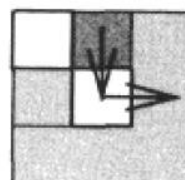
(i)



(j)



(k)



(l)



区域或背景像素

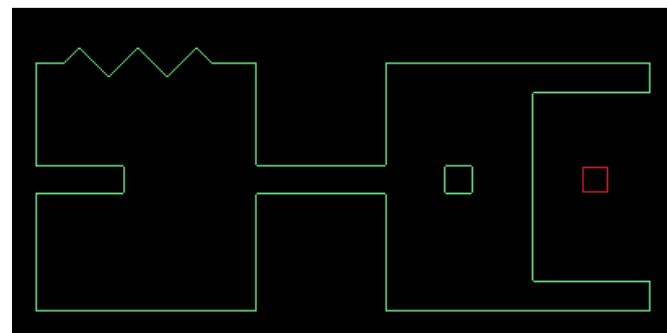
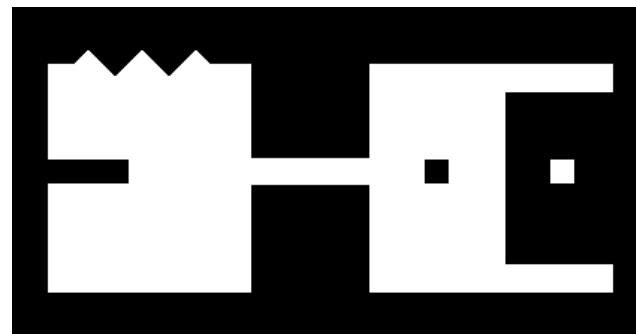
4.2 二值图像边界跟踪

➤ OpenCV:

- `findContours`: 在二值图像中获取轮廓
- `drawContours`: 在图像中绘制轮廓

```
findContours(img, contours, CV_RETR_EXTERNAL, CV_CHAIN_APPROX_NONE);
// 获取最大轮廓
if (contours.size() != 0) // 找到轮廓
{
    vector< vector<Point> >::const_iterator it = contours.begin();

    vector< vector<Point> >::const_iterator maxContourIt;
    double maxArea = -1.0;
    while(it != contours.end()) // 处理每一个轮廓
    {
        double area = contourArea(*it);
        if (area > maxArea)
        {
            maxContourIt = it;
            maxArea = area;
        }
        it++;
    }
}
```



4.3 二值图像形态学

- 数学形态学（图像代数）是以形态为基础对图像进行分析的数学工具
- 基本思想：用具有一定形态的结构元素去度量和提取图像中的对应形状，以达到图像分析和识别的目的
- 形态学图像处理的目的：
 - 图像预处理（去噪、简化形状）
 - 增强物体结构（抽取骨骼、凸包、细化、粗化）
 - 从背景中分割物体
 - 物体量化描述（面积、周长、投影）

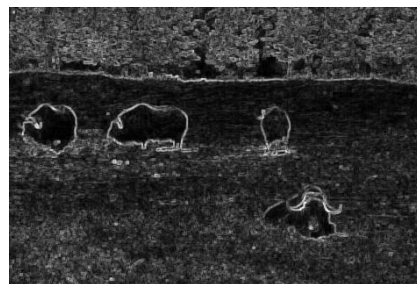
4.3 二值图像形态学

➤ 数学形态学

- 二值图像形态学
- 灰度图像形态学

➤ 二值图像形态学

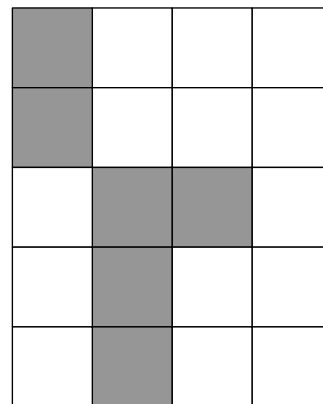
- 4.3.1 形态学的基本概念
- 4.3.2 膨胀与腐蚀
- 4.3.3 击中击不中变换
- 4.3.4 开运算与闭运算
- 4.3.5 骨架与形态学重构



4.3.1 形态学的基本概念

- 前提假设：真实图像可以表示为任意维度的点集
- 在处理二值图像形态学时，为整数对的集合 ($\in \mathbb{Z}^2$)
- 在处理灰度图像形态学时，为整数三元组的集合 ($\in \mathbb{Z}^3$)
- 图像中属于物体的点（黑色点）构成集合X，补集 X^C 对应背景（白色点）
- 集合差(difference):

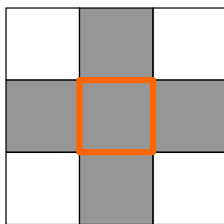
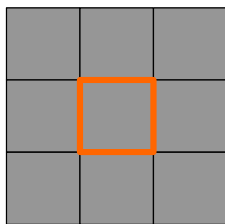
$$X \setminus Y = X \cap Y^C$$



4.3.1 形态学的基本概念

➤ 形态学变换(Morphological transformation)

- 由图像A和结构元素(Structure element)B间的关系定义
- B表示为一个关于局部原点O的邻域



- 将形态学变换 $\psi(A)$ 作用于图像A：用结构元素B扫描整幅图像，图像A和结构元素B在当前位置的作用结果保存到输出图像
- 形态学运算的对偶性：由集合补运算推导出来。对于形态学变换 $\psi(A)$ ，存在一个对偶变换 $\psi^*(A)$ ，满足：

$$\psi(A) = [\psi^*(A^c)]^c$$

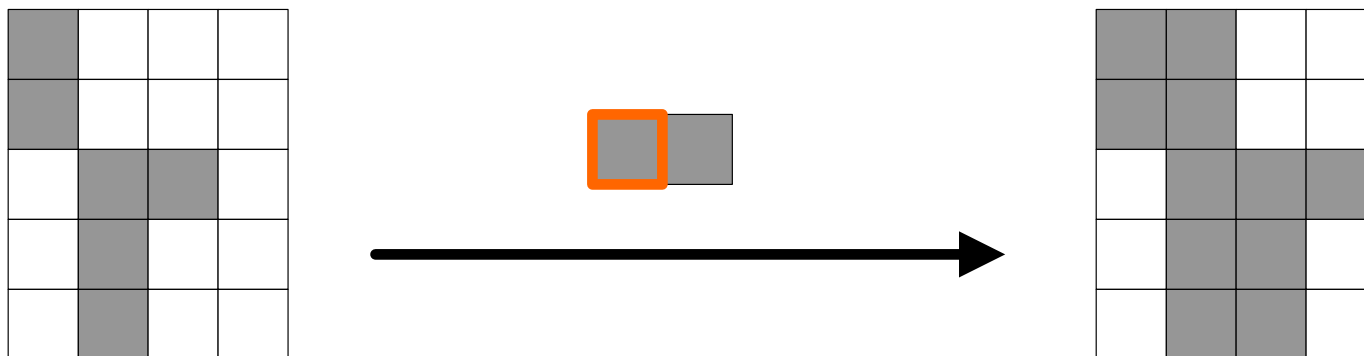
4.3.2 膨胀与腐蚀

- 膨胀在二值图像中是“加长”或“变粗”的操作
- “加长”或“变粗”的程度由结构元素控制

$$A \oplus B = \{p \in \varepsilon^2, p = a + b, a \in A \text{ 且 } b \in B\}$$

↙
↘

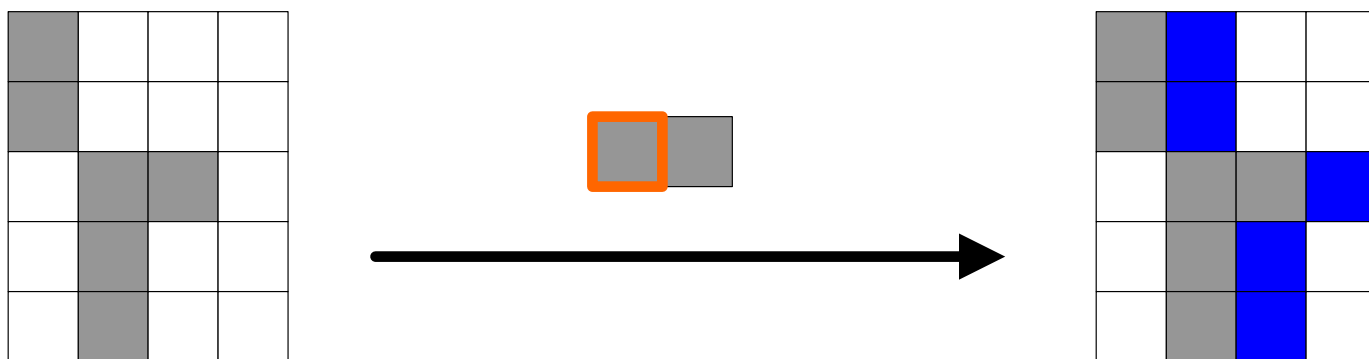
二值图像
结构元素



4.3.2 膨胀与腐蚀

➤ 膨胀的实现过程

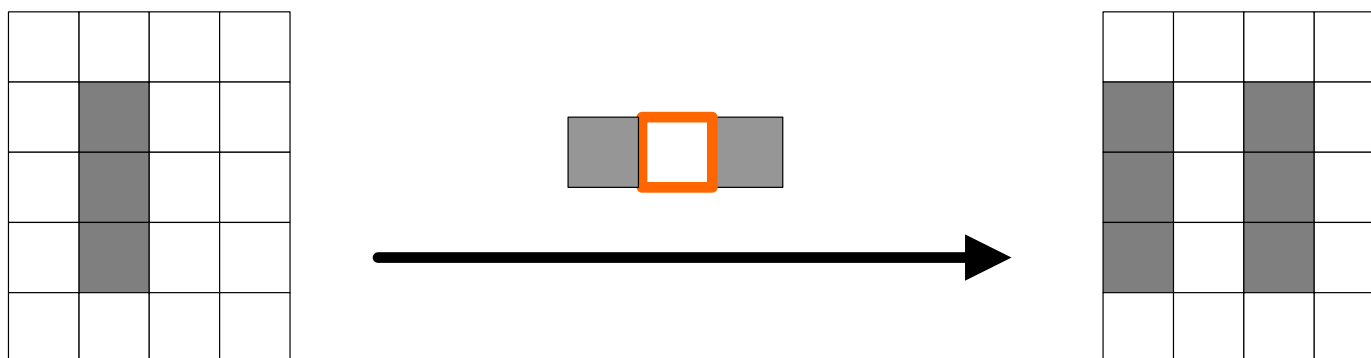
- 将结构元素B的原点移至集合A的某一点
- 将结构元素B中的点与该点相加，得到对集合中该点的膨胀结果
- 对集合A中所有元素重复该过程



4.3.2 膨胀与腐蚀

➤ 膨胀的实现过程

- 将结构元素B的原点移至集合A的某一点
- 将结构元素B中的点与该点相加，得到对集合中该点的膨胀结果
- 对集合A中所有元素重复该过程



残缺文本的膨胀

Matlab

```
img = imread('text.tif');  
element = [0 1 0; 1 1 1; 0 1 0];  
dilated = imdilate(img,element);
```

OpenCV

```
Mat element = (Mat_<uchar>(3, 3)  
    << 0, 1, 0, 1, 1, 1, 0, 1, 0);  
Mat dilated;  
dilate(img, dilated, element);
```

Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

year

Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

year



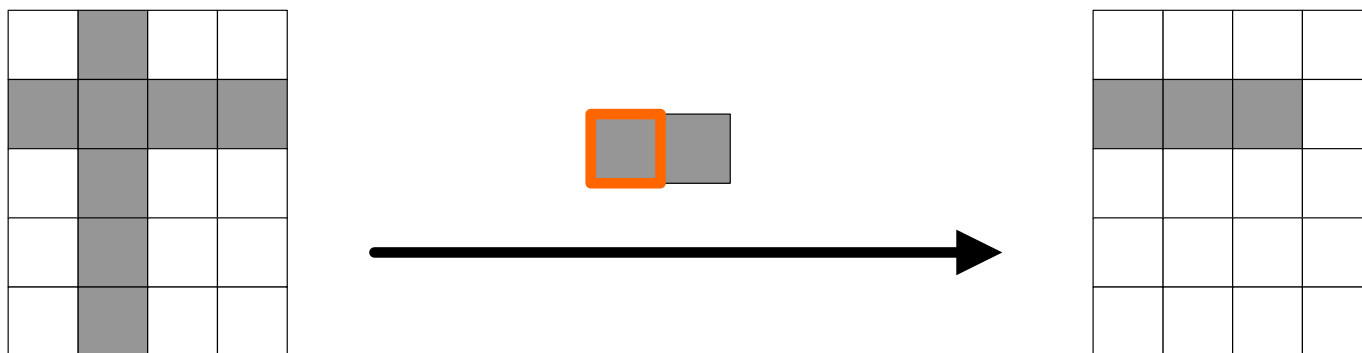
0	1	0
1	1	1
0	1	0

4.3.2 膨胀与腐蚀

- 腐蚀在二值图像中是“收缩”或“细化”的操作
- “收缩”或“细化”的程度由结构元素控制

$$A \ominus B = \{p \in \varepsilon^2, p + b \in A, \forall b \in B\}$$

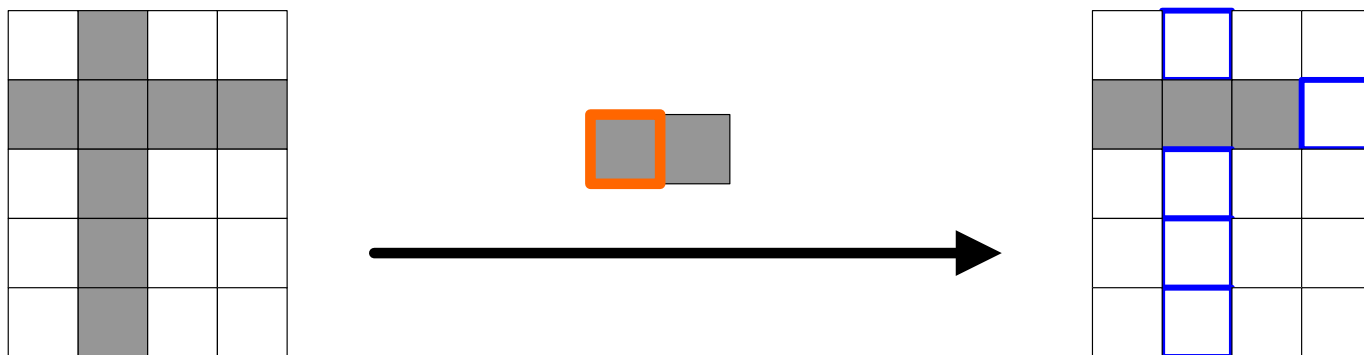
二值图像
结构元素



4.3.2 膨胀与腐蚀

➤ 腐蚀的实现过程

- 将结构元素B的原点移至集合A的某一点
- 若结构元素B属于集合A，则保留该点作为腐蚀结果
- 对集合A中所有元素重复该过程



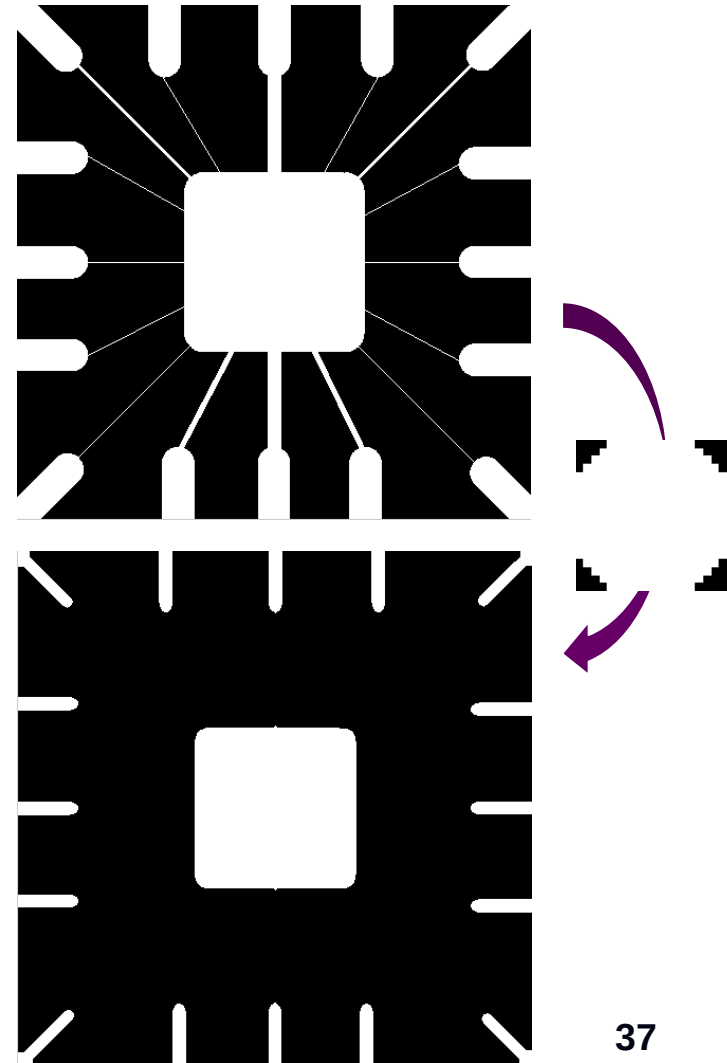
细线删除

Matlab

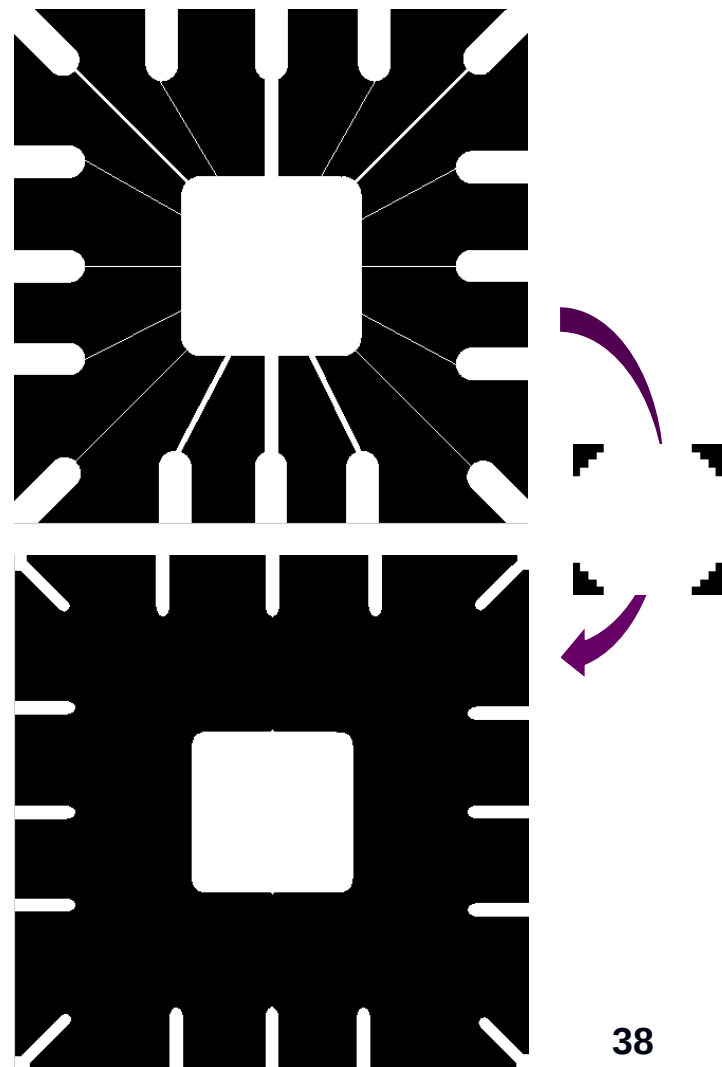
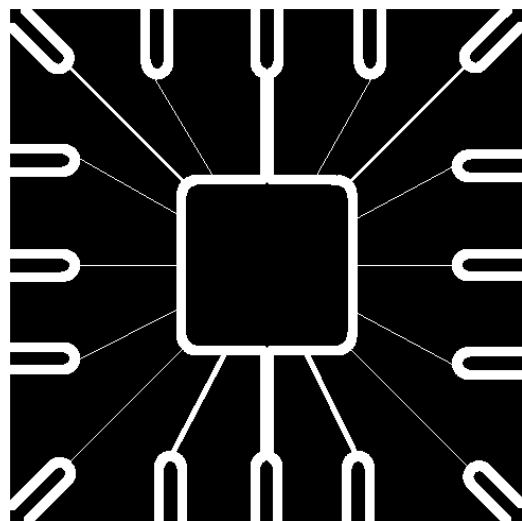
```
img = imread('wirebond.tif');  
% 创建半径为10的圆盘型结构元素  
element = strel('disk',10);  
eroded = imerode(img, element);
```

OpenCV

```
Mat element = getStructuringElement  
    (MORPH_ELLIPSE, Size(19, 19));  
Mat eroded;  
erode(img, eroded, element);
```



细线删除



举例：膨胀腐蚀综合运用



二值图像



原图膨胀



膨胀结果面积处理



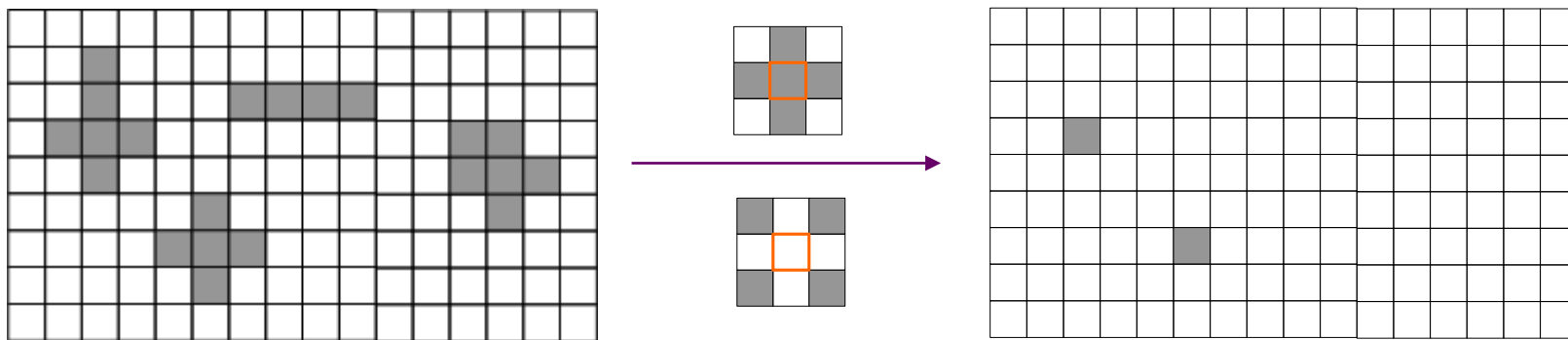
腐蚀结果

4.3.3 击中击不中变换

- 击中击不中变换用来查找像素局部模式

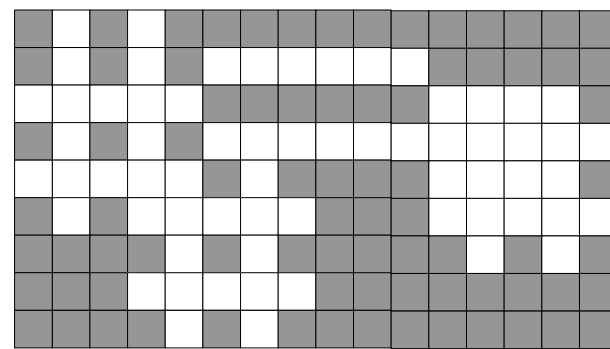
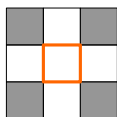
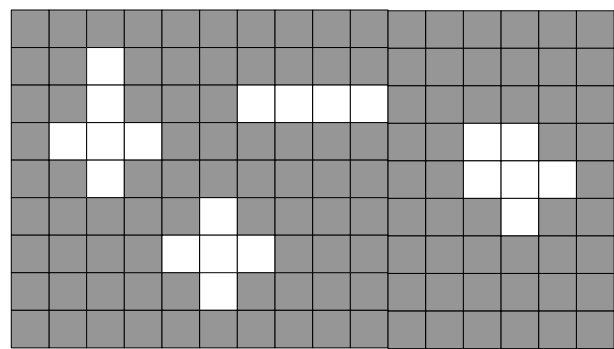
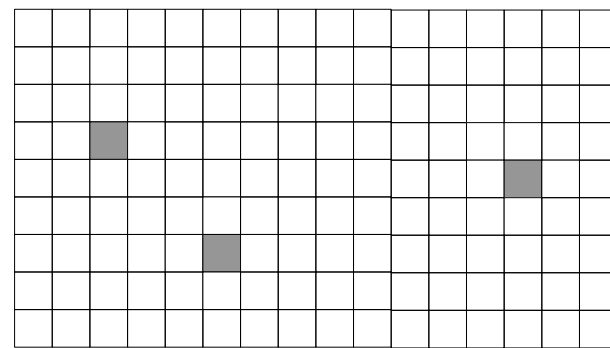
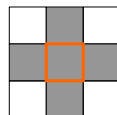
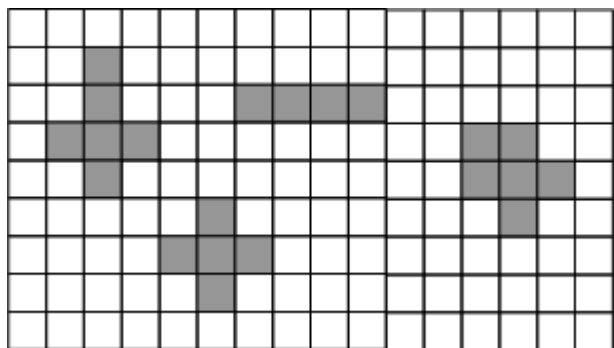
$$A \otimes B = \{p : B_1 \subset A \text{ 且 } B_2 \subset A^c\}$$

$B = (B_1, B_2)$ 是复合结构元素， B_1, B_2 不相交



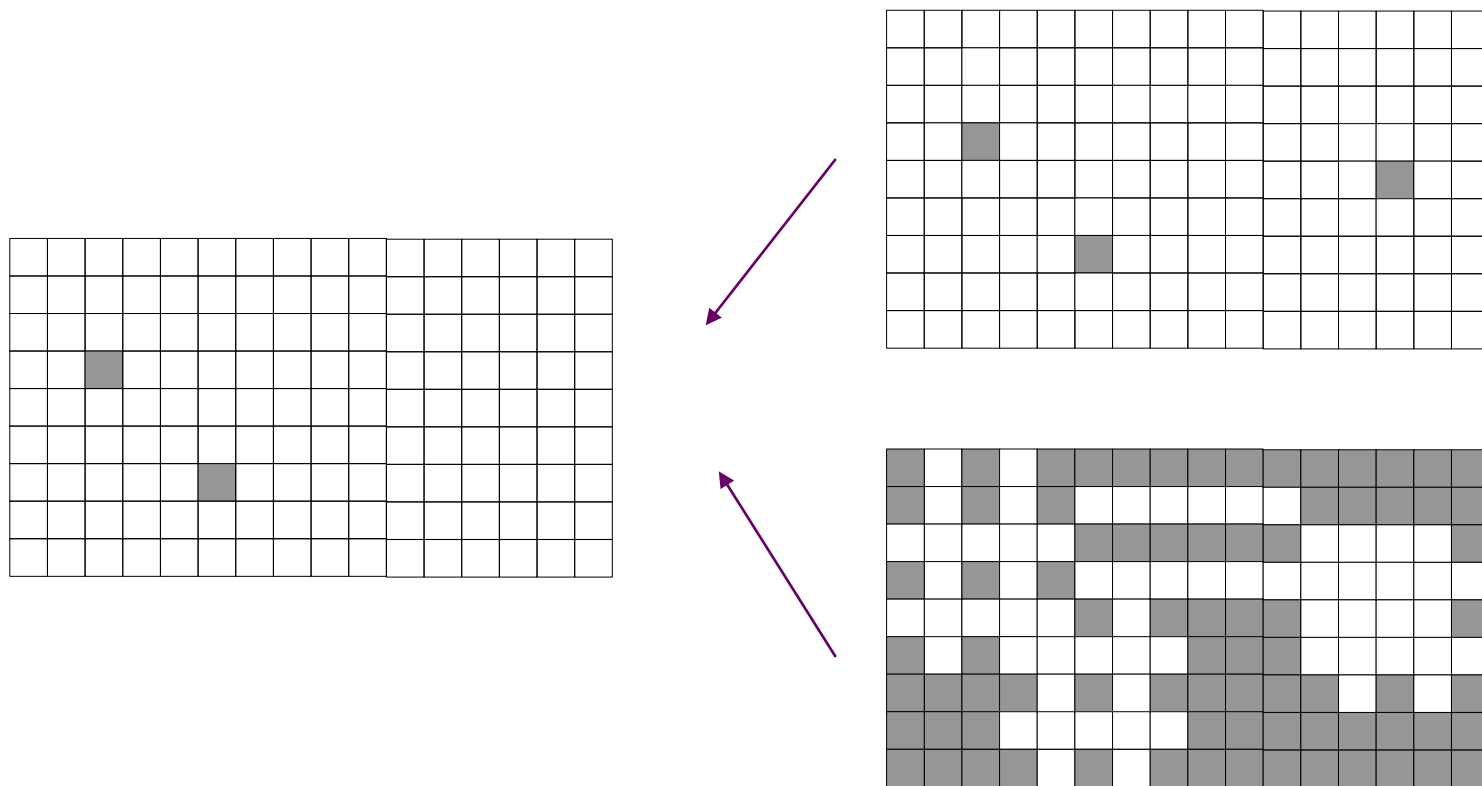
4.3.3 击中击不中变换

$$A \otimes B = (A \ominus B_1) \cap (A^c \ominus B_2)$$



4.3.3 击中击不中变换

$$A \otimes B = (A \ominus B_1) \cap (A^c \ominus B_2)$$



4.3.3 击中击不中变换

➤ Matlab: bwhitmiss: 两种参数

- $BW2 = bwhitmiss(BW1, SE1, SE2)$
- $BW2 = bwhitmiss(BW1, INTERVAL)$

等同于 $BW2 = bwhitmiss(BW1, INTERVAL==1, INTERVAL==-1)$

➤ OpenCV: morphologyEx

◆ morphologyEx()

```
void cv::morphologyEx ( InputArray  src,
                        OutputArray dst,
                        int          op, MORPH_HITMISS
                        InputArray  kernel,
                        Point        anchor = Point(-1,-1) ,
                        int          iterations = 1 ,
                        int          borderType = BORDER_CONSTANT ,
                        const Scalar & borderValue = morphologyDefaultBorderValue()
                        )
```

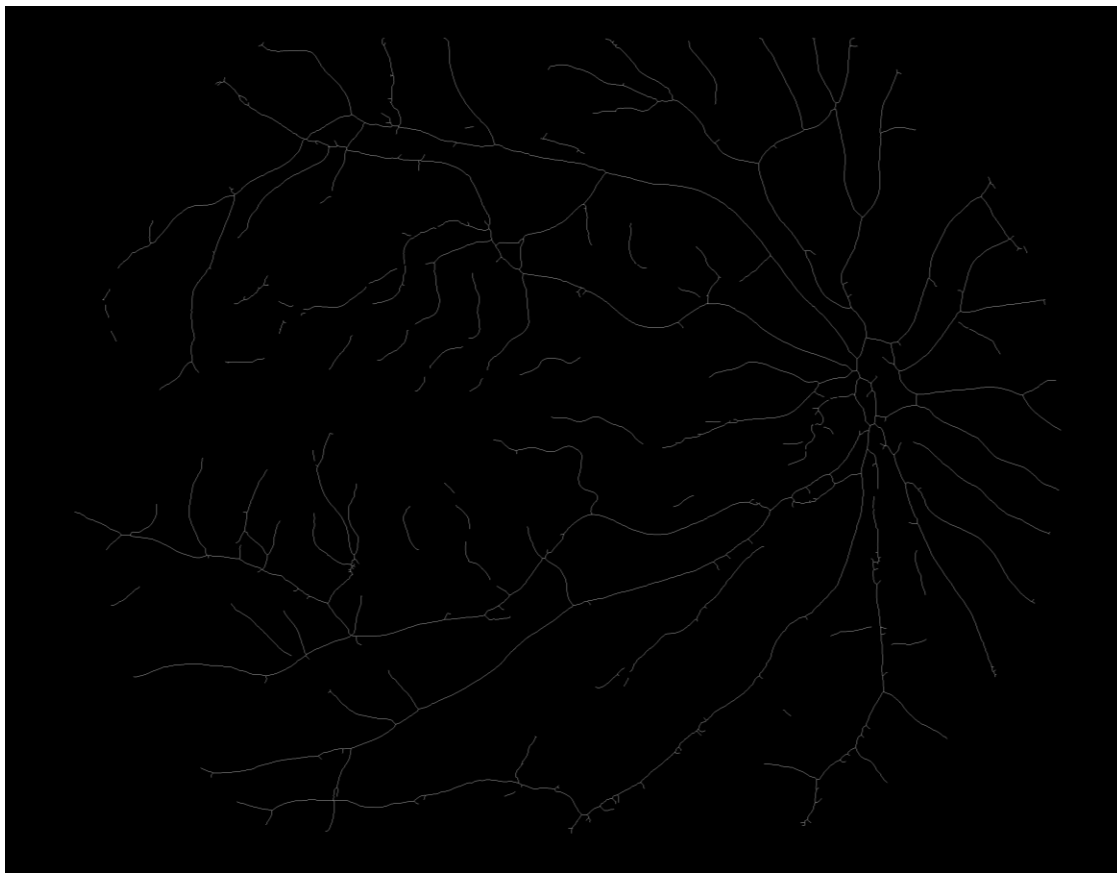
0	1	0
1	0	1
0	1	0

0	0	0
0	1	0
0	0	0

0	1	0
1	-1	1
0	1	0

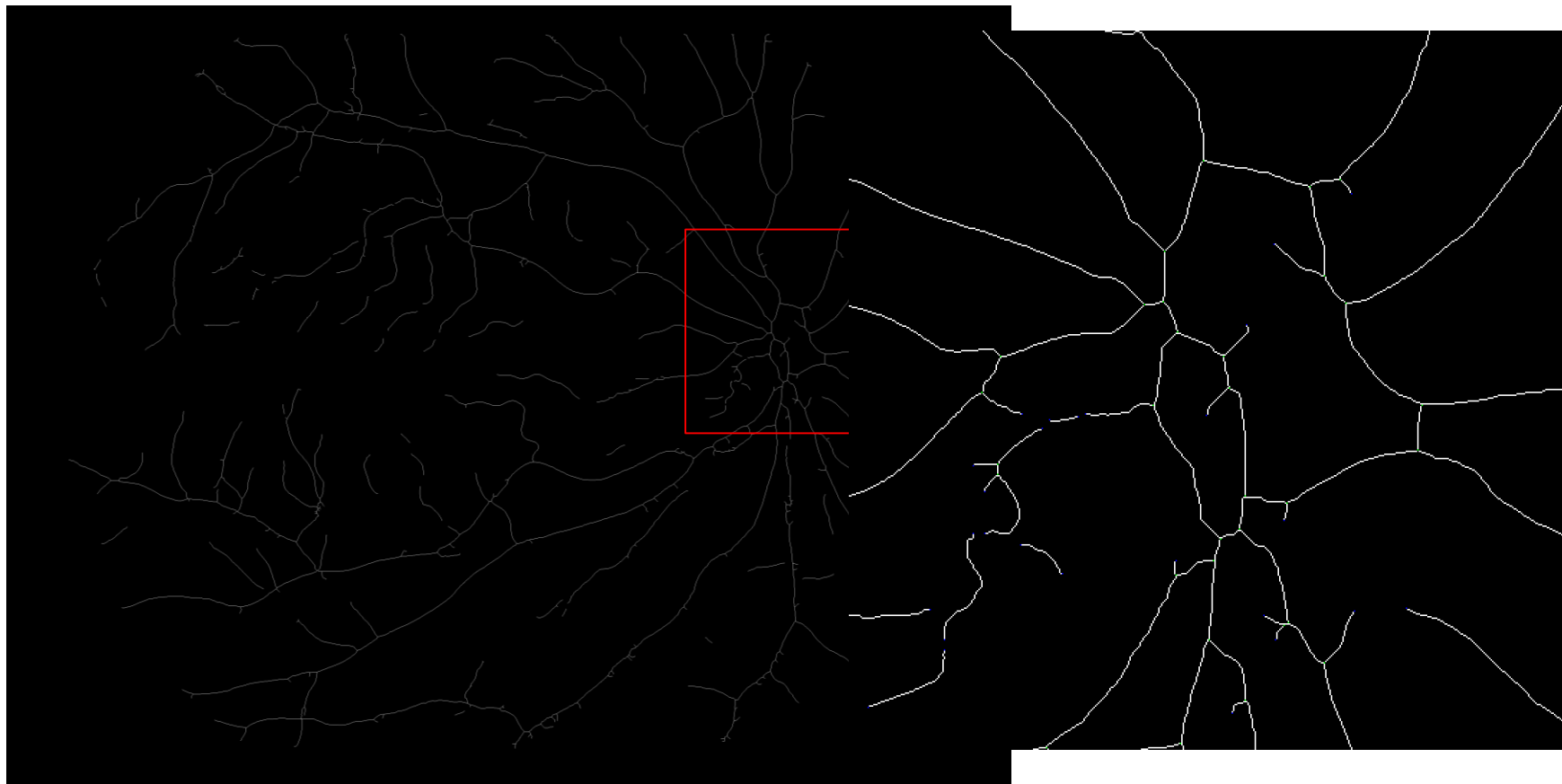
击中击不中变换举例

- 在血管网络中提取曲线的端点和线交叉点



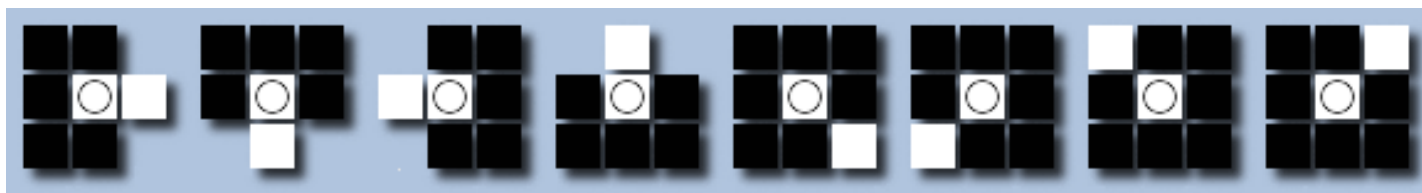
击中击不中变换举例

- 在血管网络中提取曲线的端点和线交叉点

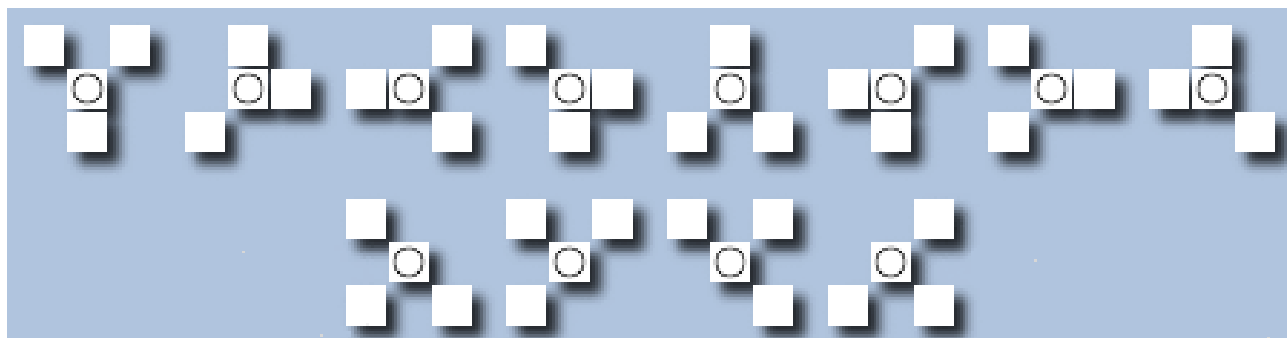


击中击不中变换举例

- 在血管网络中提取曲线的端点和线交叉点
 - 提取端点的结构元素组合

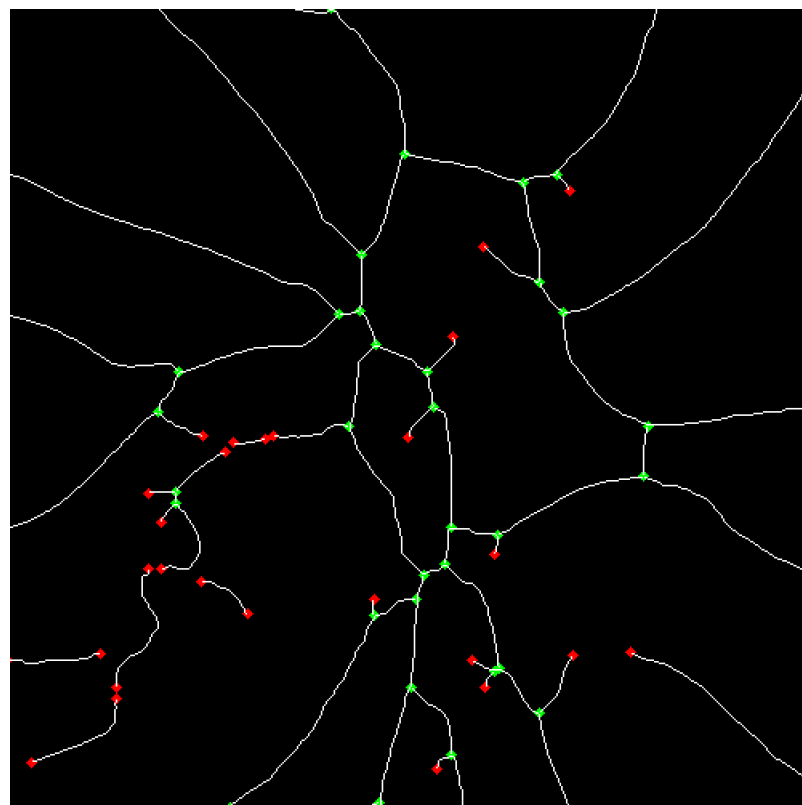
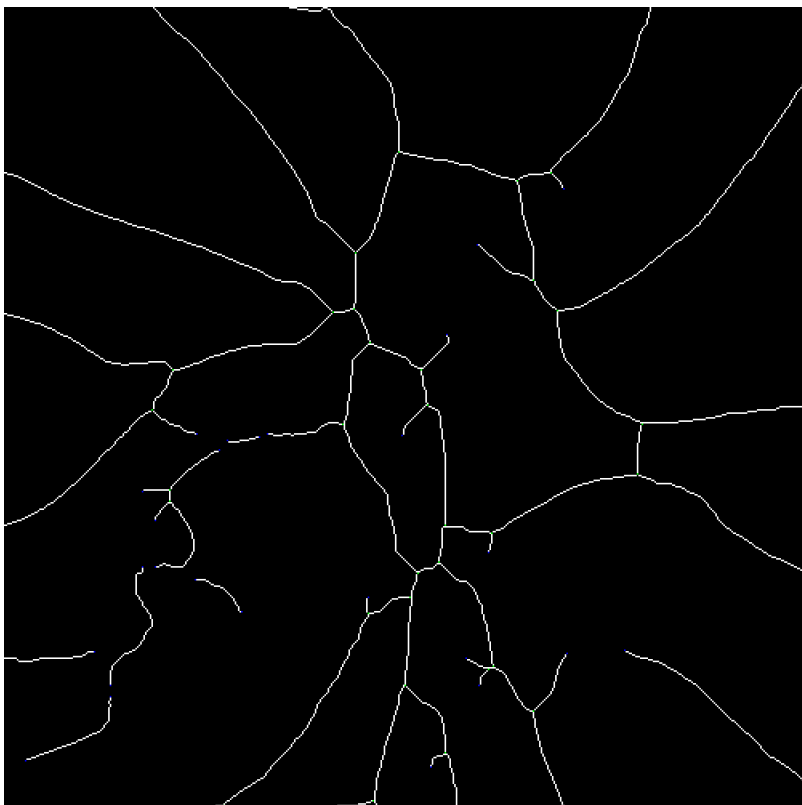


- 提取线交叉点的结构元素组合



击中击不中变换举例

- 在血管网络中提取曲线的端点和线交叉点



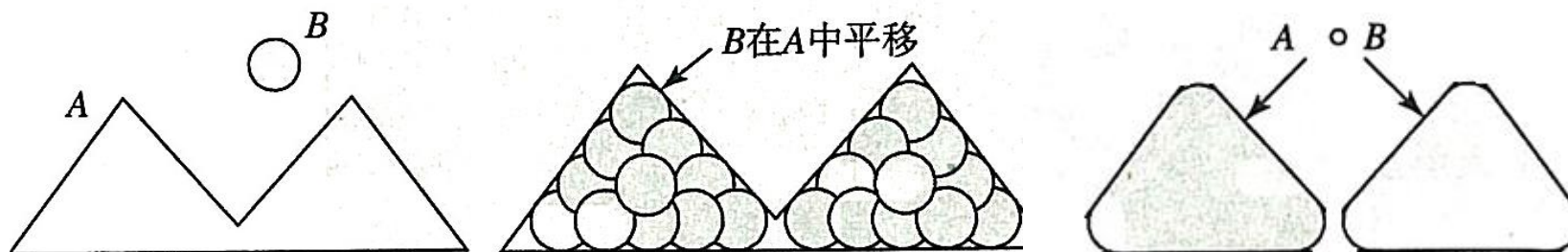
4.3.4 开运算与闭运算



- 开运算：先腐蚀再膨胀

$$A \circ B = (A \ominus B) \oplus B$$

- 开运算是结构元素B在集合A内完全匹配的并集，完全删除了不能包含结构元素的对象区域
- 开运算使图像的轮廓变得光滑，断开狭窄的间断，去掉细小的突出物



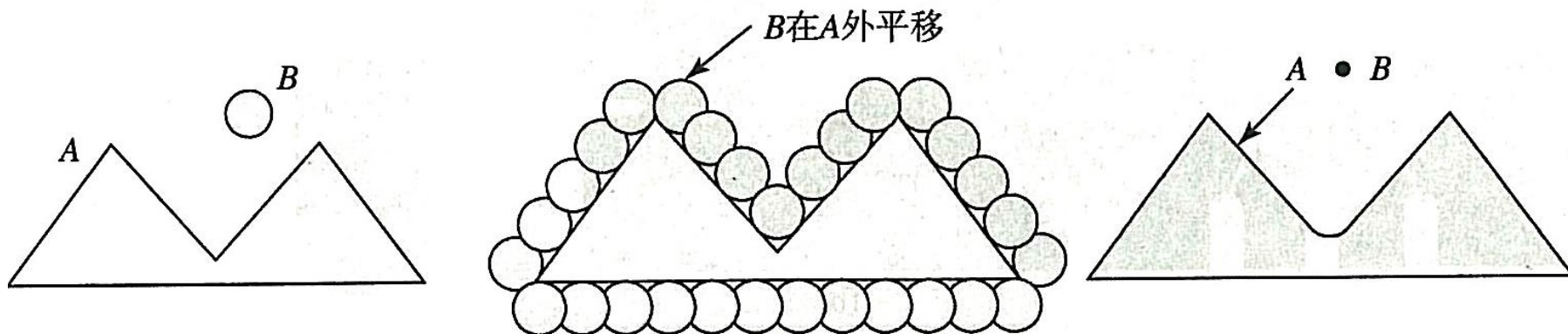
4.3.4 开运算与闭运算



- 闭运算：先膨胀再腐蚀

$$A \bullet B = (A \oplus B) \ominus B$$

- 闭运算：结构元素B在集合A外侧平移，这些平移的并集就是闭运算的结果
- 闭运算使图像的轮廓变得光滑，但与开运算不同的是，它会将狭窄的缺口连接形成细长的弯口，并填充比结构元素小的洞



4.3.4 开运算与闭运算

Matlab

```
f = imread('shapes.tif');  
se = strel('square',20);  
fc = imclose(f,se);  
fo = imopen(f,se);  
foc = imclose(fo,se);
```

OpenCV

```
Mat se = Mat::ones(20, 20, CV_8U);  
Mat fc, fo;  
morphologyEx(f, fc, MORPH_CLOSE, se);  
morphologyEx(f, fo, MORPH_OPEN, se);
```

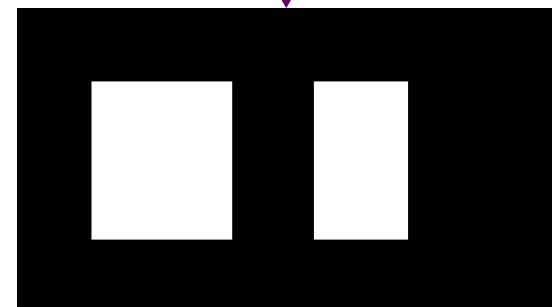


闭运算

开运算



闭运算



其它形态学运算

➤ 形态学梯度

- 膨胀结果与腐蚀结果的差

$$A \oplus B - A \ominus B$$

➤ 顶帽变换

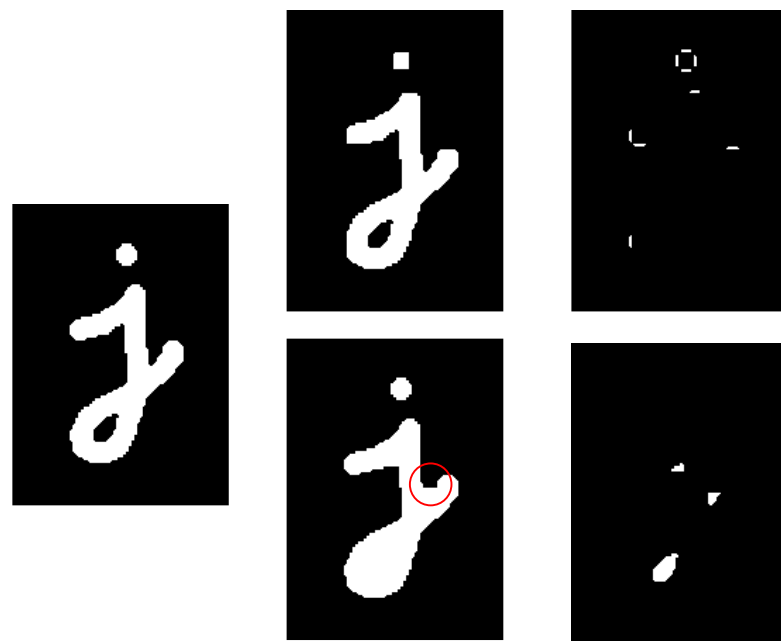
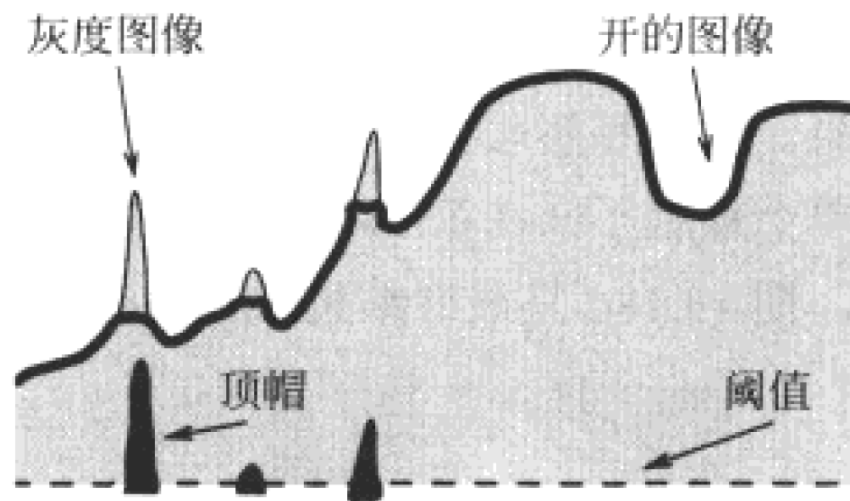
- 原图像与开运算的差

$$A - A \circ B$$

➤ 黑帽变换

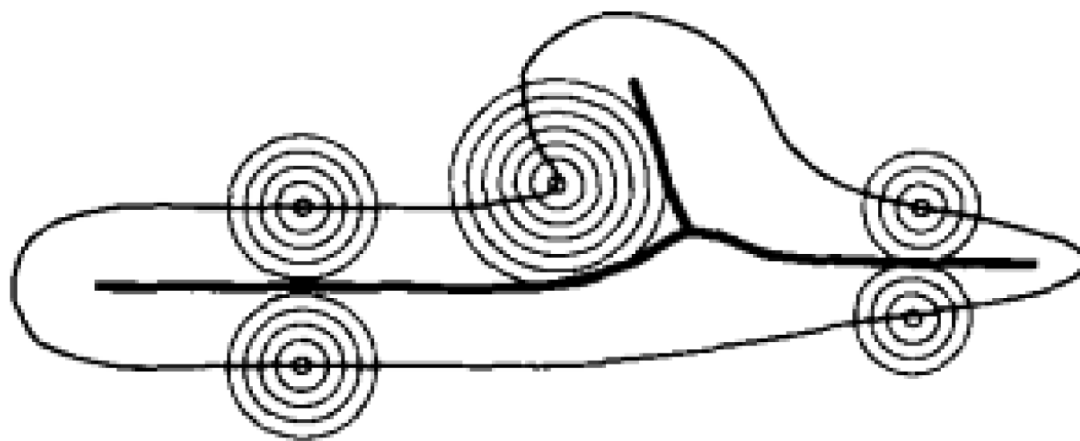
- 闭运算与原图像的差

$$A \bullet B - A$$



4.3.5 骨架与形态学重构

- (1) 骨架、中轴与最大球
 - 中轴变换 (medial axis transformation) : 在一个区域 (点集) 的边界的所有地方同时点燃草地火焰, 且假设火焰以相同的速度向区域内部传递
 - 骨架 (skeleton) 为两个或更多火焰前沿相遇点的集合

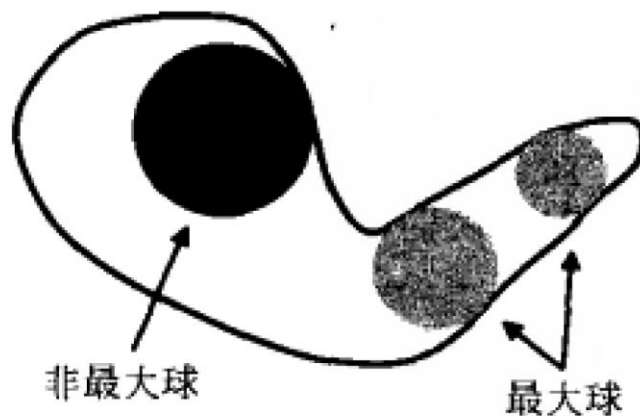
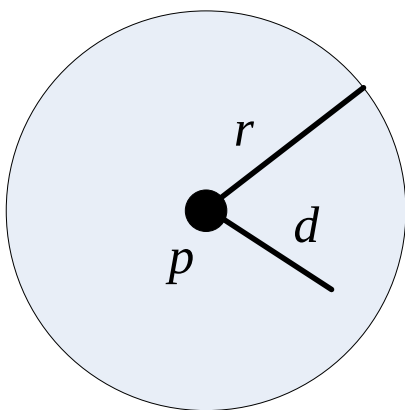


(1) 骨架、中轴与最大球

➤ 最大球

- 一个中心为 p 、半径为 r 的球 $B(p, r)$ 为距点 p 的距离 d 小于等于 r 的所有点的集合
- 称集合 X 中的一个球 B 是最大的，当且仅当 X 中没有能够包含 B 的球，即对任意 B' ，满足：

$$B \subseteq B' \subseteq X \Rightarrow B' = B$$



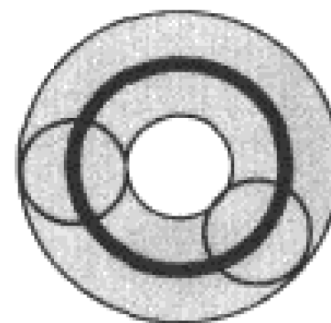
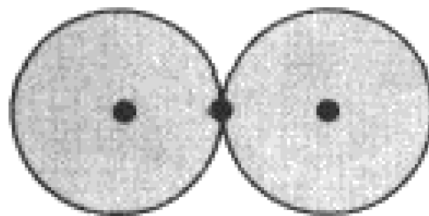
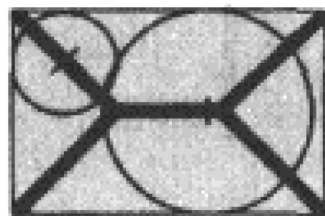
(1) 骨架、中轴与最大球

➤ 最大球骨架 (skeleton by maximal ball)

- 集合 $X \subset \mathbb{R}^2$ 的最大球骨架 $S(X)$ 为所有最大球的圆心 p 组成的集合:

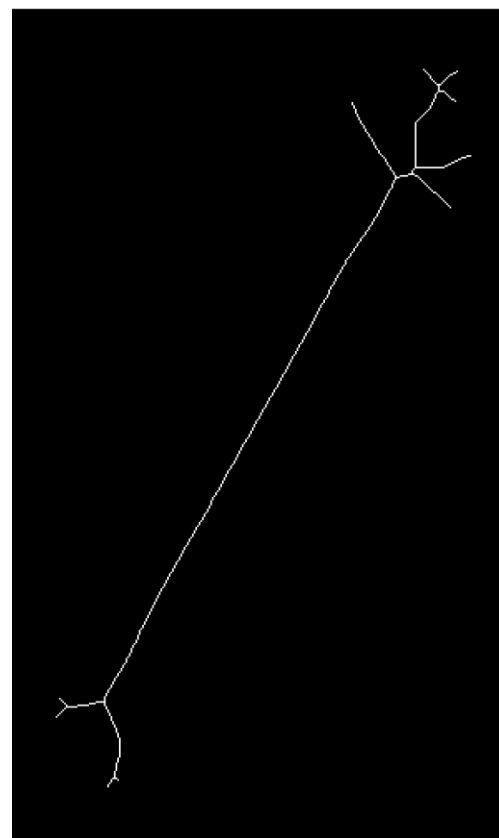
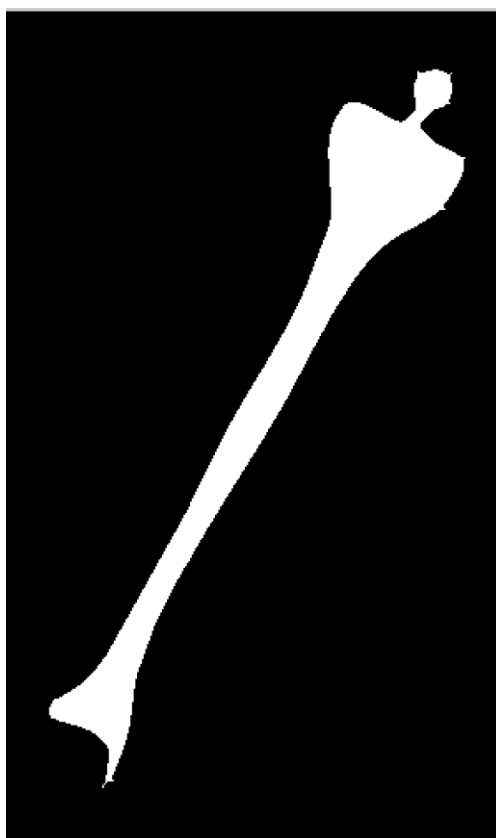
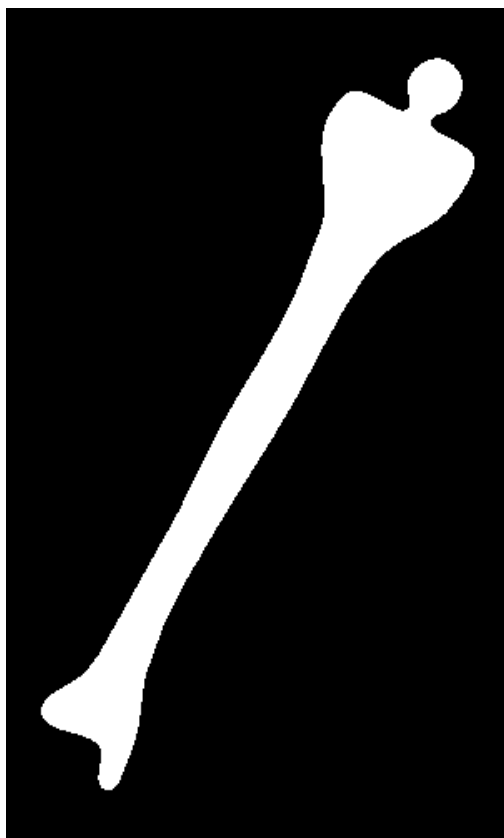
$$S(X) = \{p \in X : \exists r \geq 0, B(p, r) \text{ 是 } X \text{ 的一个最大球}\}$$

- 圆盘的骨架退化为其圆心
- 两端为圆形的带状区域的骨架位于其内部中心的单位宽度线段



(1) 骨架、中轴与最大球

➤ Matlab: bwmorph(img,'skel',n)



(1) 骨架、中轴与最大球

➤ 基于顶帽变换的简单骨架检测

➤ 伪代码

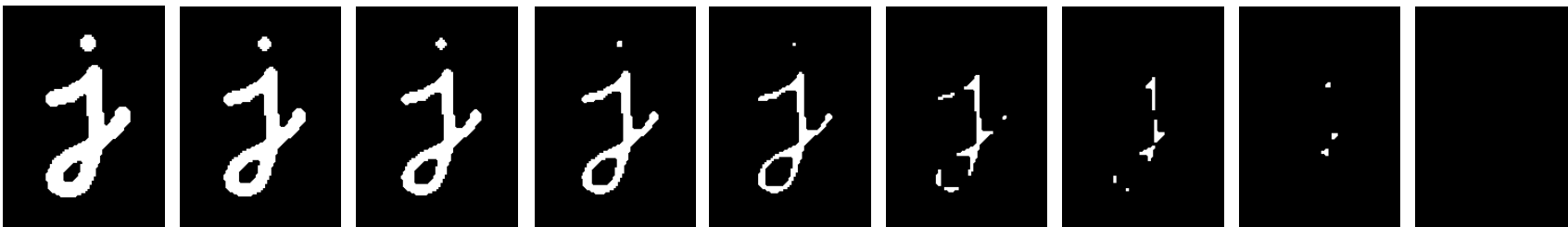
```

1. while (not_empty(img))
2. {
3.     skel = skel | (img - open(img));
4.     img = erosion(img);
5. }
    
```

骨架



开运算



(2) 细化与粗化

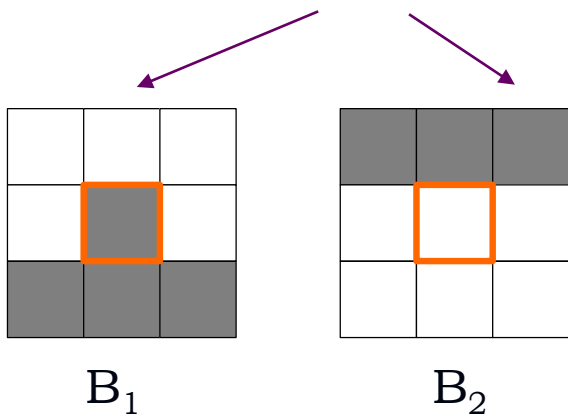
- 细化 (thinning) 与粗化 (thickening)
 - 对于图像A和复合结构元素 $B = (B_1, B_2)$
 - 细化: $A \oslash B = X \setminus (X \otimes B)$
 - 粗化: $A \odot B = X \cup (X^c \otimes B)$
- 连续使用细化和粗化
 - 复合结构元素序列 $\{B_{(i)}\} = \{B_{(1)}, B_{(2)}, \dots, B_{(n)}\}$, $B_{(i)} = (B_{i_1}, B_{i_2})$
 - 顺序细化: $A \oslash \{B_{(i)}\} = (((A \oslash B_{(1)}) \oslash B_{(2)}) \dots \oslash B_{(n)})$
 - 顺序粗化: $A \odot \{B_{(i)}\} = (((A \odot B_{(1)}) \odot B_{(2)}) \dots \odot B_{(n)})$
 - 细化和粗化的顺序变换最终收敛到某个图像，迭代次数依赖于图像中的物体和所用结构元素，若迭代过程中连续两幅图像相同，则细化/粗化结束。

(2) 细化与粗化

➤ 复合结构元素序列

- Golay字符集：由某个结构元素在数字光栅下的旋转生成
- 结构元素L

$$L_1 = \begin{bmatrix} 0 & 0 & 0 \\ * & 1 & * \\ 1 & 1 & 1 \end{bmatrix}, L_2 = \begin{bmatrix} * & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & * \end{bmatrix}, L_3 = \begin{bmatrix} 1 & * & 0 \\ 1 & 1 & 0 \\ 1 & * & 0 \end{bmatrix}, \dots$$

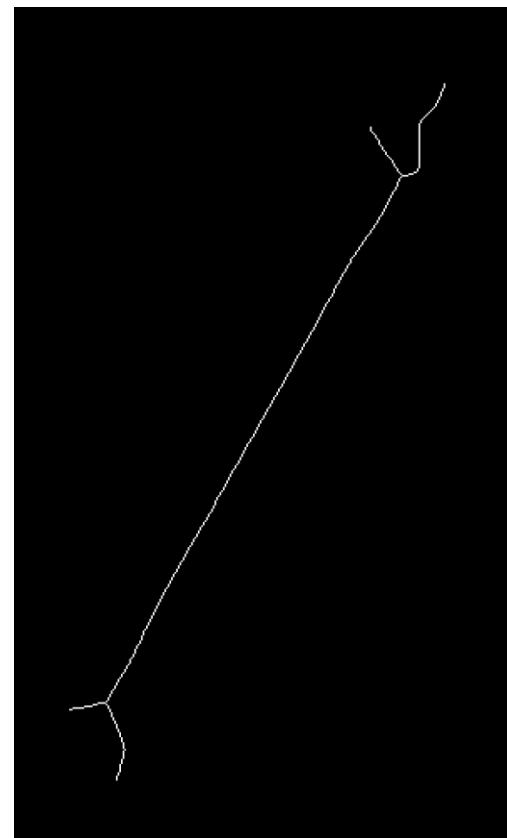
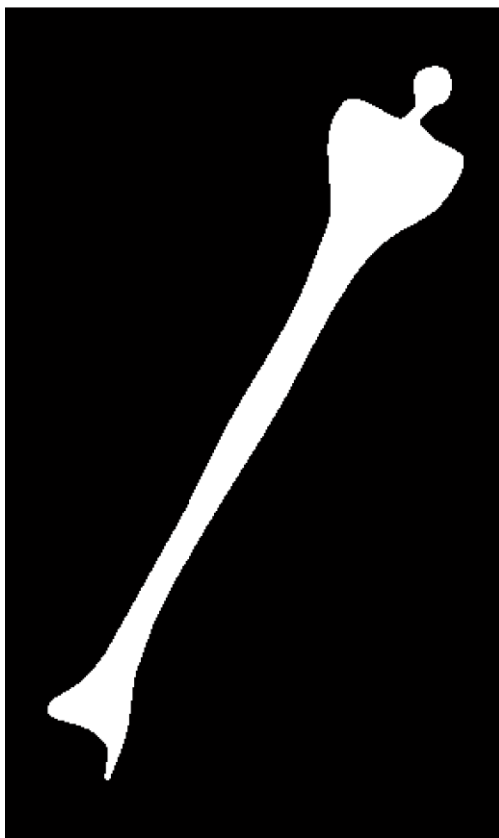
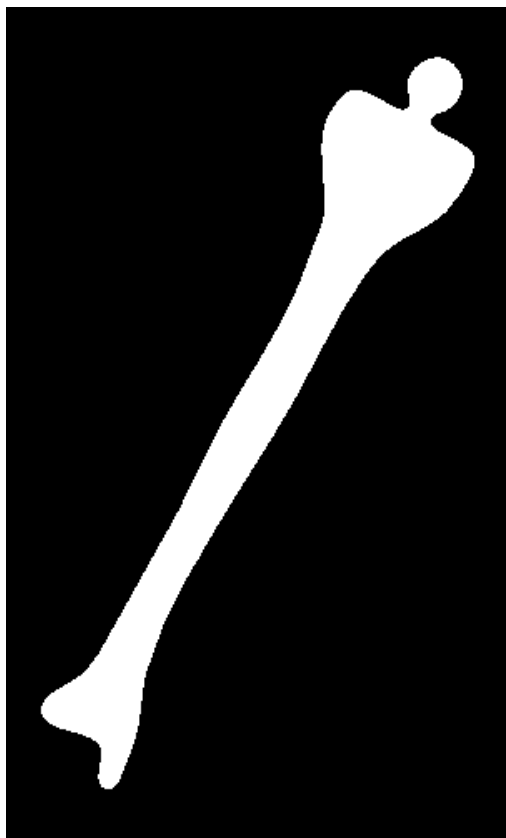


安工大 安工大

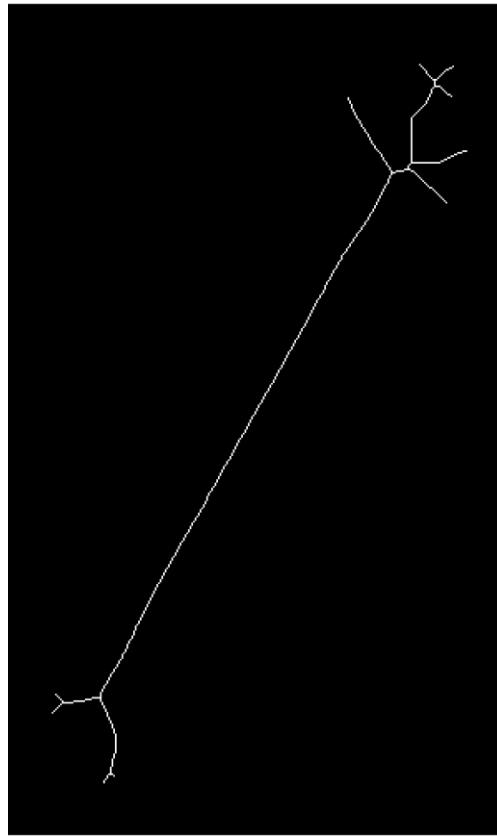
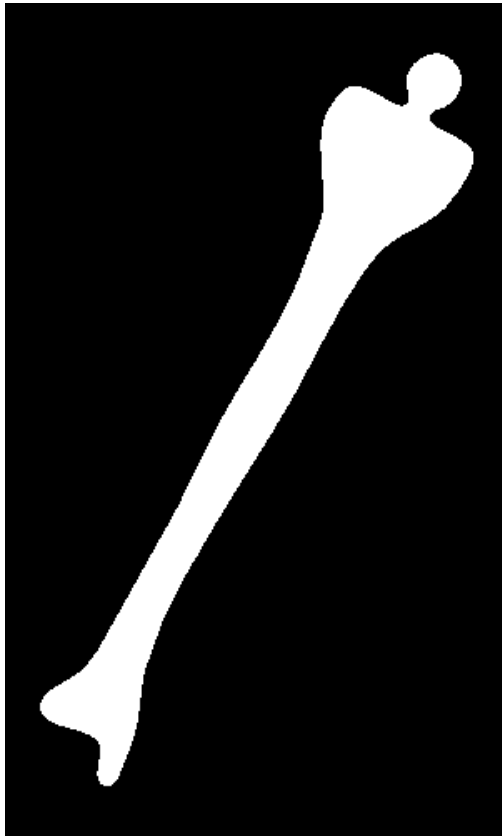
刘宏申. 击中击不中变换在笔画细化中的应用.
安徽工业大学学报：自然科学版, 2002, 19(3):3.

(2) 细化与粗化

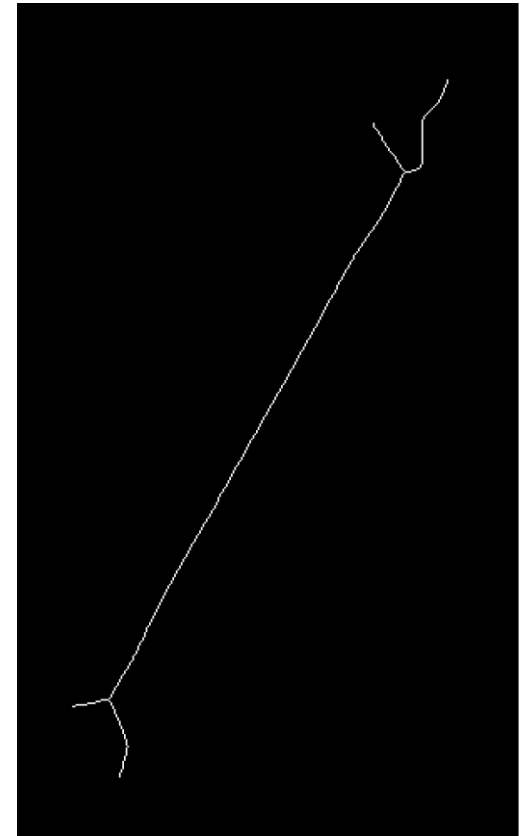
➤ Matlab: `bwmorph(img,'thin',n)`



(2) 细化与粗化



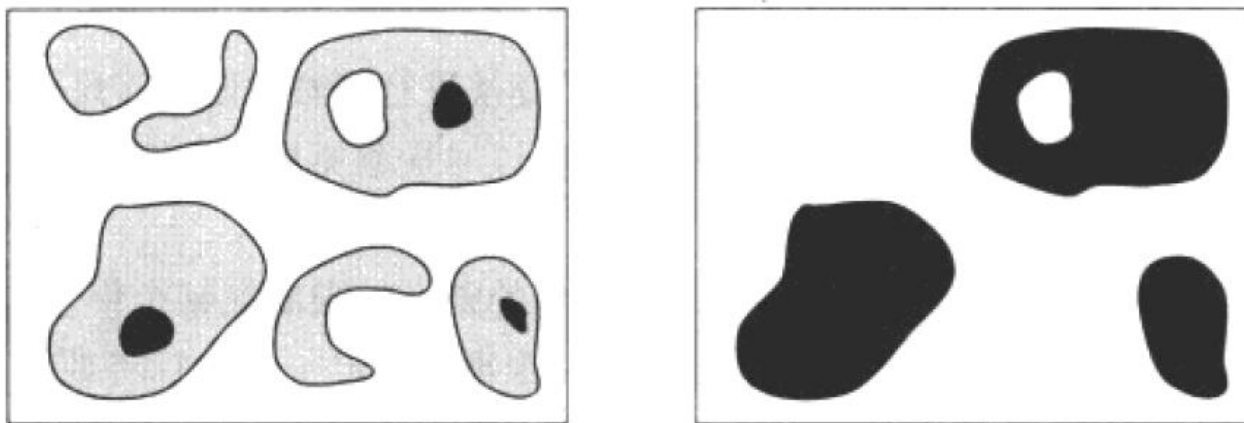
骨架



细化

(3) 形态学重构

- 从一幅二值化图像中重构出一个给定形状的物体



- 重构涉及两幅图像和一个结构元素
 - 标记(marker): 变换的开始点
 - 掩模(mask): 约束变换过程
 - 结构元素: 定义邻接性 (默认: 8-邻接, 3×3 的矩阵)

(3) 形态学重构

➤ 形态学重构的过程

g 是掩模图像， f 是标记，标记 f 必须是 g 的子集
从 f 重构 g 记为 $R_g(f)$:

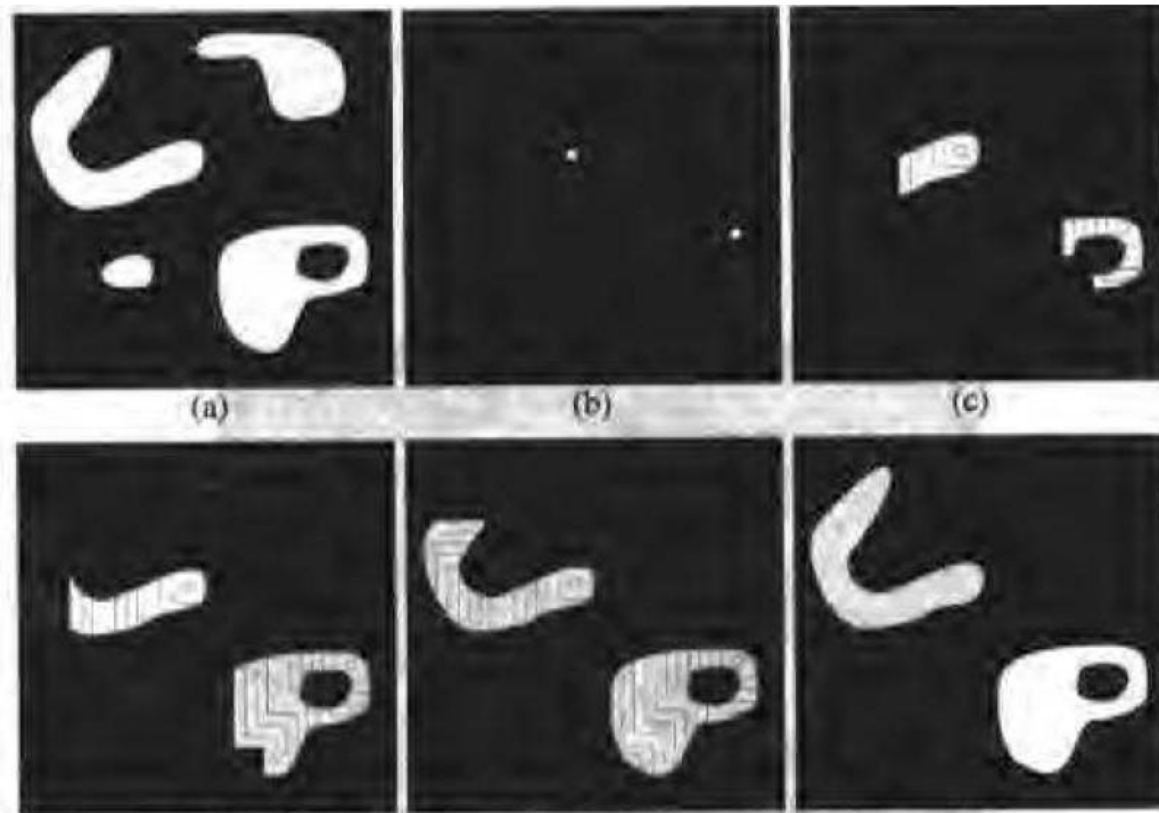
1. 将 h_1 初始化为标记图像 f
2. 创建3*3的结构元素 B
3. 重复

$$h_{k+1} = (h_k \oplus B) \cap g$$

直到 $h_{k+1} = h_k$

(3) 形态学重构

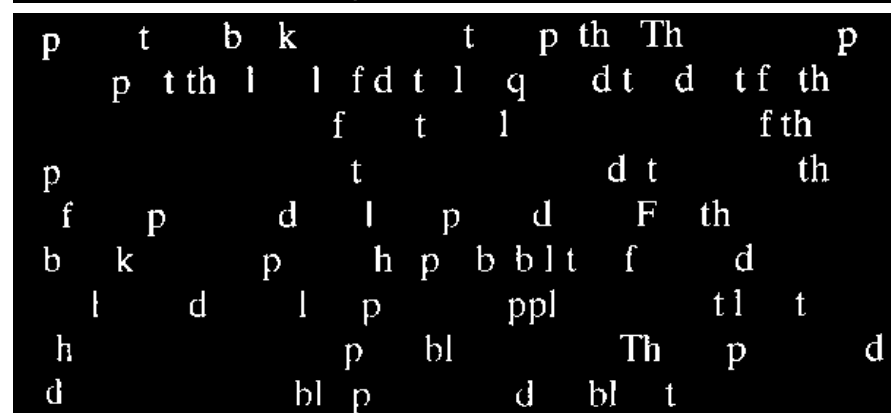
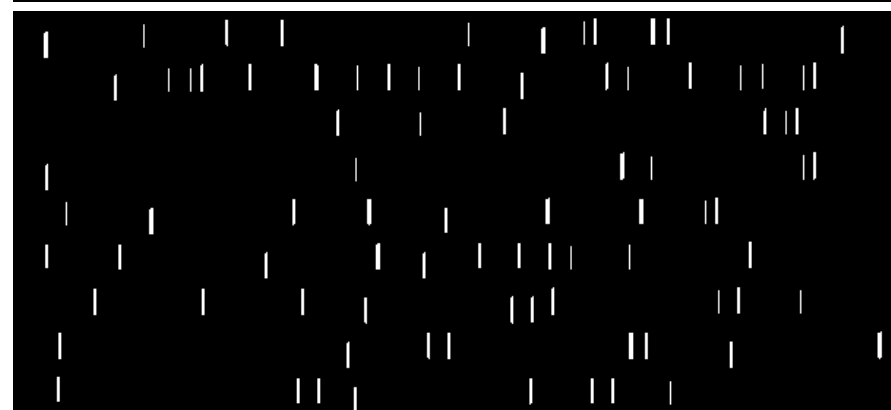
➤ 形态学重构的过程



由重构做开运算

Matlab

```
f = imread('text.tif');  
fe=imerode(f,ones(51,1));  
fe=imdilate(fe,ones(51,1));  
fobr=imreconstruct(fe,f);
```



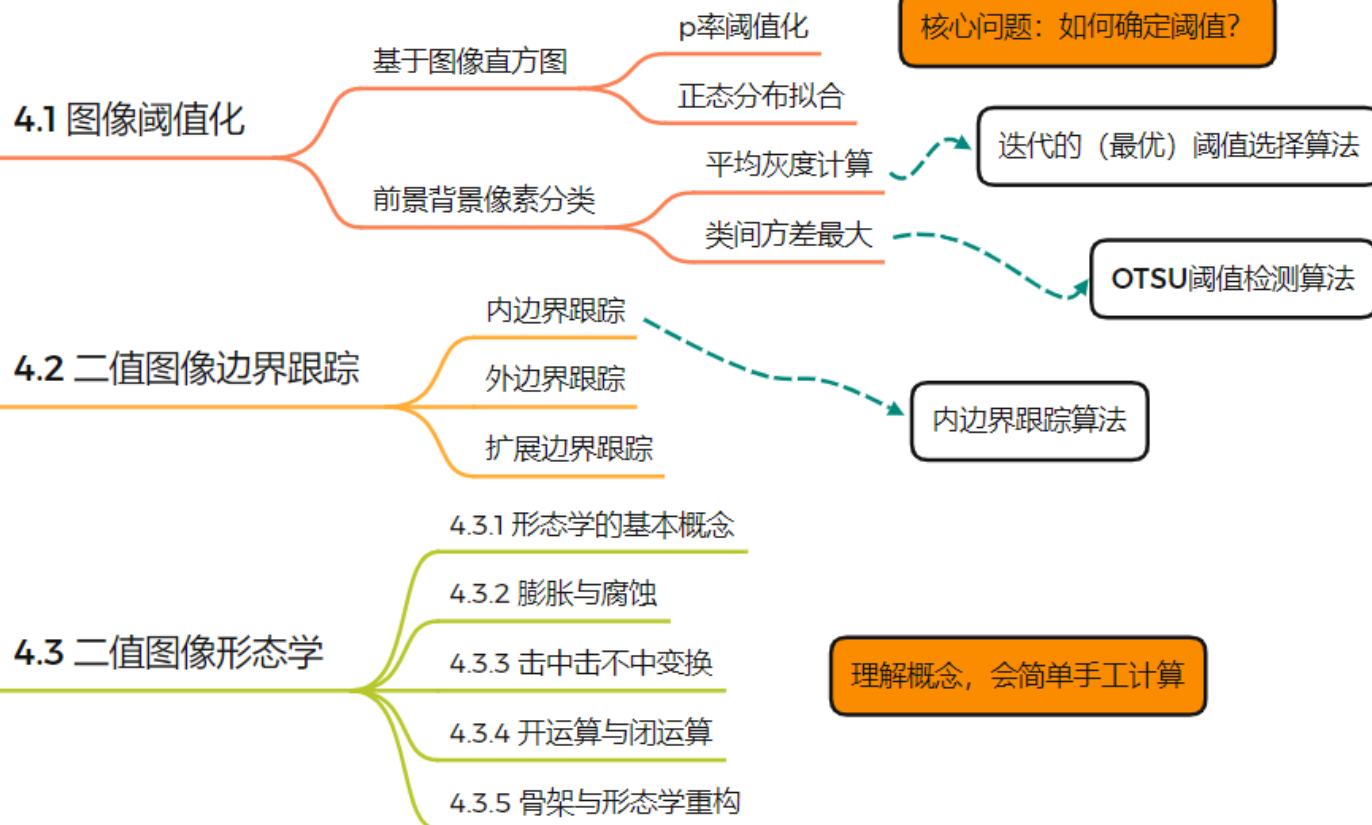
ponents or broken connection paths. There is no point past the level of detail required to identify those components.

Segmentation of nontrivial images is one of the most difficult tasks in image processing. Segmentation accuracy determines the effectiveness of computerized analysis procedures. For this reason, considerable effort must be taken to improve the probability of rugged segmentation. In applications such as industrial inspection applications, at least some degree of segmentation is possible at times. The experienced image processing designer invariably pays considerable attention to such details.

总结



二值图像处理



The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學
Nankai University